

MODULE 7

Introduction to Replication and Sharding

How replica sets keep training_store online through a failure — and how sharding lets its orders grow past one server.

Availability vs scale

Replica sets and failover

Read and write guarantees

Sharded clusters and keys

Routing and design

LEARNING OBJECTIVES

What you will understand by the end of this module.



BY THE END OF THIS MODULE, YOU CAN

1. Tell availability from scalability
2. Explain replica-set roles, the oplog, failover
3. Choose basic read and write settings
4. Describe the parts of a sharded cluster
5. Evaluate shard keys and query routing
6. Decide when to replicate, shard, or both



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHY

do we need copies and pieces?

HOW

does a replica set survive failure?

WHICH

member serves a read, and when?

WHAT

makes up a sharded cluster?

HOW

do we choose a shard key?

WHEN

do we replicate, shard, or both?

KEY TAKEAWAY Replication keeps the data available; sharding distributes the data and the work.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 **Replica set**

A group of mongod servers holding the same data: one primary and its secondaries.

2 **Oplog**

The operation log: a capped collection of every change, which secondaries copy and apply.

3 **Election and failover**

When the primary is lost, a majority of voters elects a secondary as the new primary.

4 **Write concern**

How many members must confirm a write before the application gets an answer.

5 **Read preference and read concern**

Which member serves a read — and which data it may return.

6 **Sharded cluster**

Shards hold the data, config servers hold the metadata, mongos routes every request.

7 **Shard key**

The indexed field or fields that decide which shard stores each document.

8 **Chunk and balancer**

A chunk is one range of shard-key values; the balancer moves ranges between shards.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

CA

Canada (zone label)

EU

European Union (zone label)

RAM

Random Access Memory

COLLSCAN

Collection Scan

HA

High Availability

URI

Uniform Resource Identifier

CPU

Central Processing Unit

IXSCAN

Index Scan

US

United States (zone label)

ESR

Equality, Sort, Range

oplog

Operations log

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

FROM MODULE 6 TO MODULE 7

Our queries are fast on one server — now keep the data online and ready to grow.

MODULE 6 GAVE US

- ✓ Indexes shaped like our queries
- ✓ explain(): IXSCAN versus COLLSCAN
- ✓ The ESR guideline for compound keys
- ✓ { customerId: 1, createdAt: -1 } for history
- ✓ Evidence before changing the design

NOW Survive a server failure — and plan how orders can outgrow one server.



OUR PATH THROUGH MODULE 7

- 1 Availability versus scalability
- 2 Replica sets, the oplog and elections
- 3 Failover and what the application sees
- 4 Read preference, read and write concern
- 5 Replication lag and rollback
- 6 Sharded clusters and shard keys
- 7 Chunks, routing and the design decision

Part 1

Part 2

Part 2

Part 3

Part 3

Part 4

Part 5

KEY TAKEAWAY Module 6 made one server fast; Module 7 keeps training_store online and lets it grow.

PART 1: AVAILABILITY AND SCALE

Two different problems — and two different mechanisms.

Part 1
Availability and scale

Part 2
Replica sets

Part 3
Guarantees and lag

Part 4
Shard keys

Part 5
Routing and design

SO FAR

- ✓ Indexes make single queries fast
- ✓ explain() shows the work each query does
- ✓ training_store runs on one server so far

NEXT QUESTION

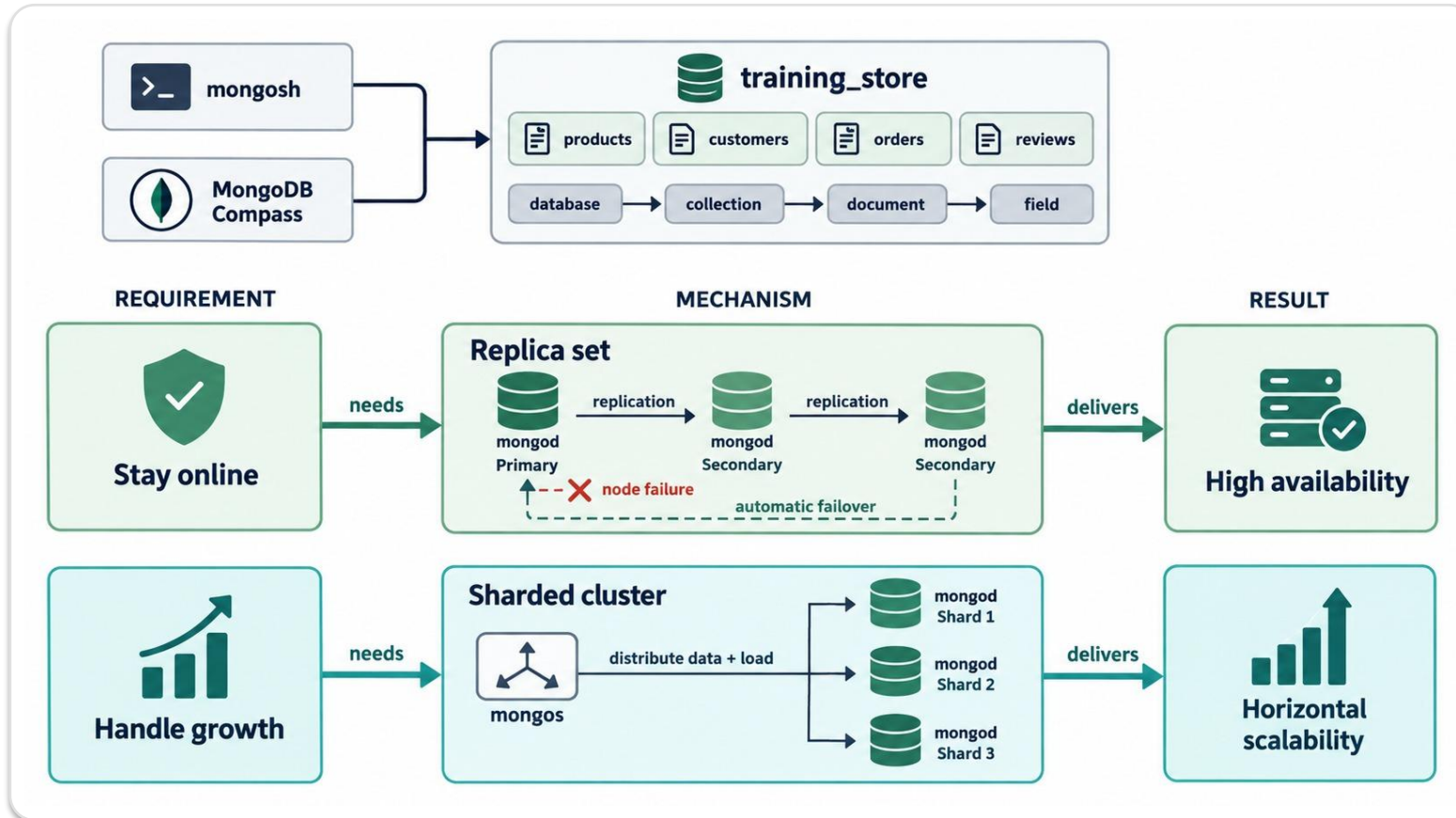
What should happen when a server fails — and when the data outgrows one server?

IN THIS PART

- 1 Requirement to mechanism
- 2 Scaling up versus scaling out
- 3 Replication versus sharding
- 4 What neither mechanism does

HIGH AVAILABILITY AND HORIZONTAL SCALABILITY

Stay online needs a replica set; handle growth needs a sharded cluster.



Stay online → replica set

Copies of the same data; if the primary fails, a secondary takes over.

Handle growth → sharded cluster

mongos spreads data and load across several shards.

Different problems

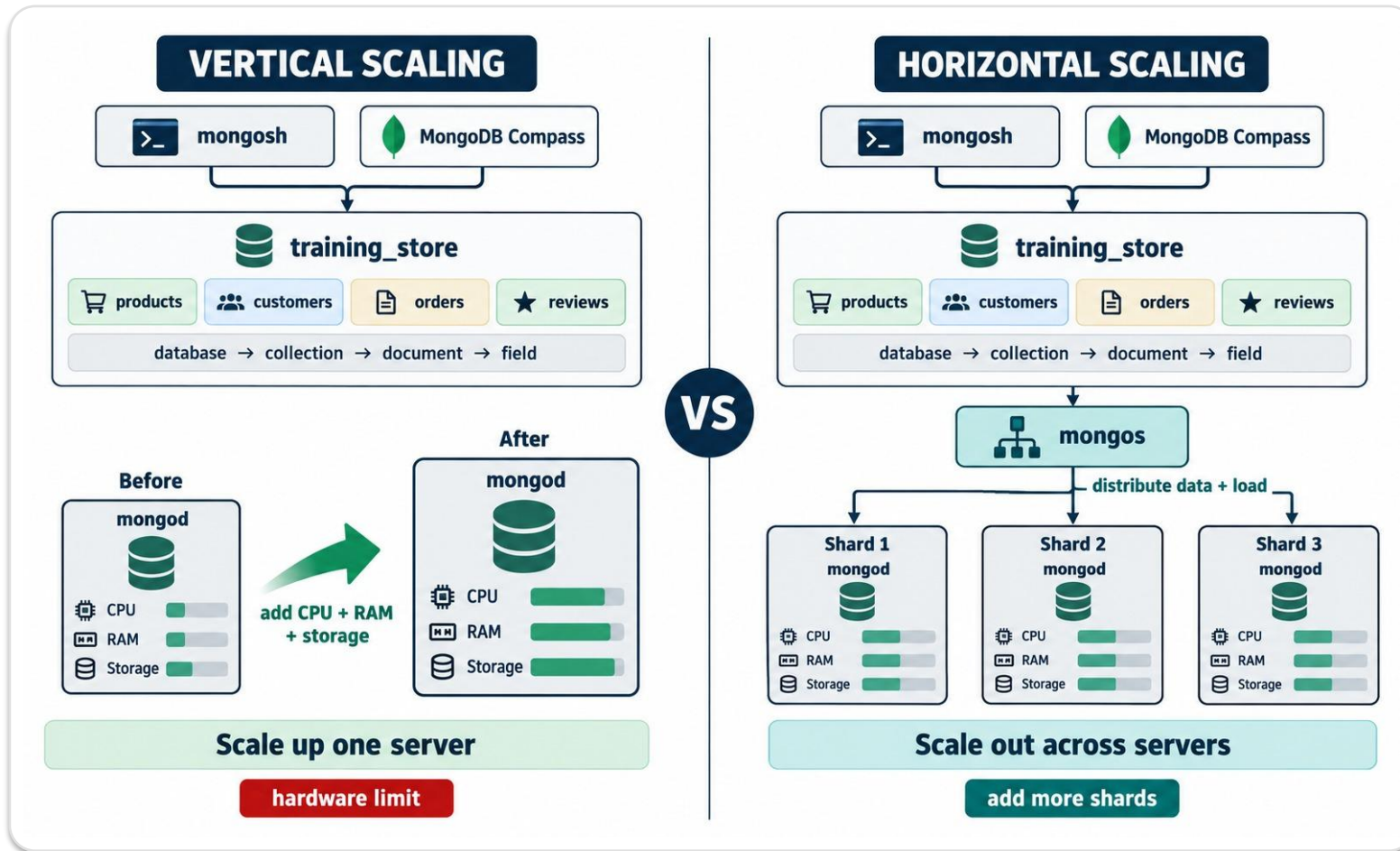
Copies add availability, not capacity. Shards add capacity, not copies.

SIMPLIFIED Each shard is drawn as one mongod; in production it is a replica set.

KEY TAKEAWAY Name the requirement first: staying online needs replication, growing needs sharding.

VERTICAL SCALING VS HORIZONTAL SCALING

Scale up one server — or scale out across servers.



Vertical: scale up

More CPU, RAM and storage on one server — simple, until the hardware limit.

Horizontal: scale out

Add shards behind mongos; each holds part of the data.

The cost of scaling out

More servers, routing, balancing and monitoring.

START SIMPLE Tune schema and indexes before you shard.

KEY TAKEAWAY Scale up while one server is enough; scale out when the data or the writes outgrow it.

REPLICATION VERSUS SHARDING

What each mechanism delivers — and what neither does.

Requirement	Replication	Sharding
Keep several copies	Yes	Not by itself
Automatic failover	Yes	Per shard, via replication
Grow data capacity	Limited	Yes
Spread write load	Not normally	Yes
Survive one server failure	Yes	Only with replicated shards
Main objective	Availability	Capacity and throughput
Undo last week's delete	No — restore a backup	No — restore a backup

Replication

Every member holds the same data — redundancy, not more room.

Sharding

Each shard holds part of a collection — room and throughput.

TOGETHER Production clusters shard the data and replicate every shard.

KEY TAKEAWAY Replication copies the data; sharding divides it — and only a backup undoes a mistake.



PART 2: REPLICA SETS AND FAILOVER

How several copies of training_store stay in step — and take over.



SO FAR

- ✓ Availability and scale are different problems
- ✓ Replication keeps copies for availability
- ✓ Sharding divides data for capacity

NEXT QUESTION

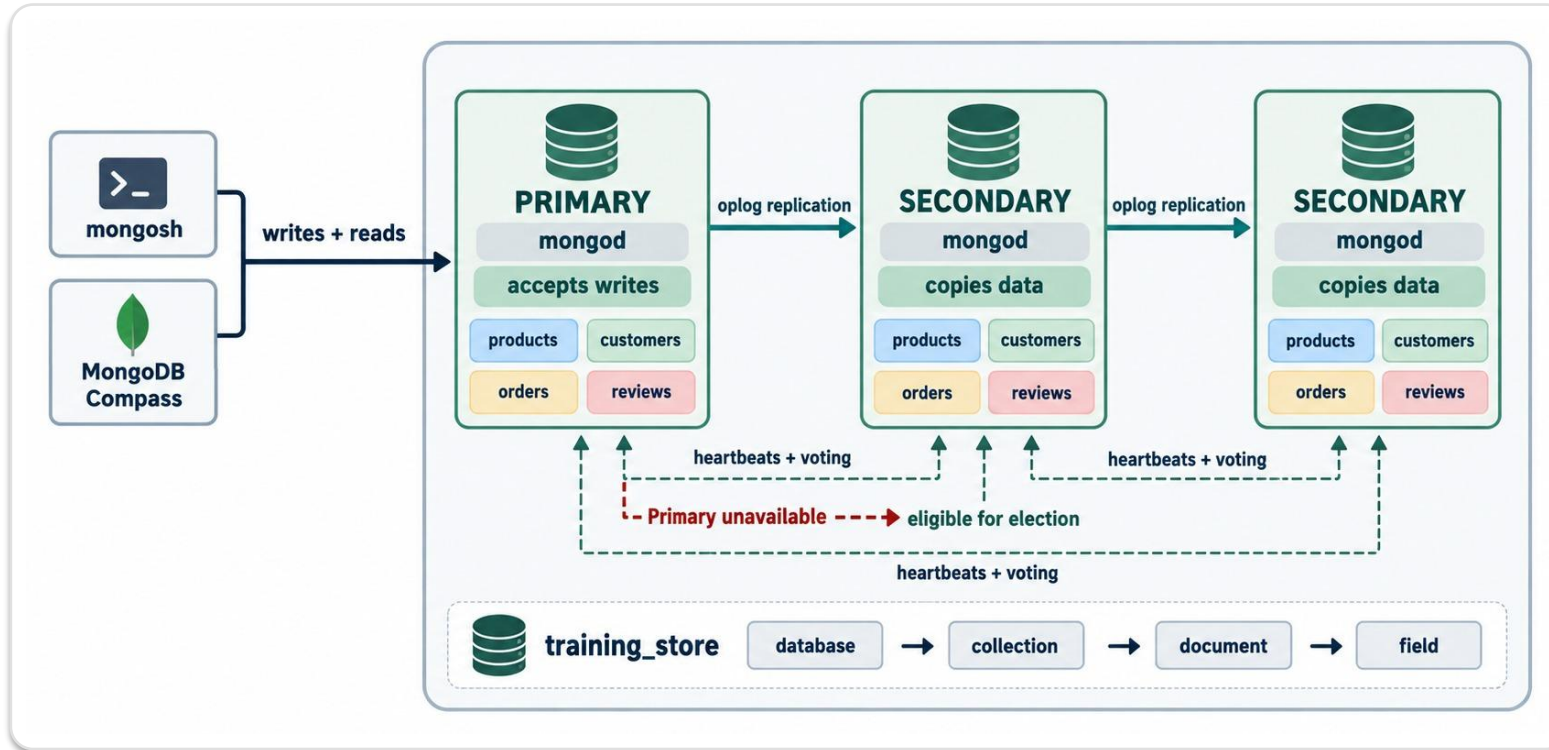
How does a replica set keep every copy in step — and pick a new primary?

IN THIS PART

- 1 Primary and secondary members
- 2 The oplog
- 3 Elections and majority
- 4 Automatic failover
- 5 What the application sees
- 6 Checking replica-set health
- 7 Exercise: count the votes

REPLICA-SET MEMBERS

One primary accepts writes; the secondaries copy the data.



Primary

The one member that accepts writes; by default it also serves reads.

Secondaries

Copy the primary's oplog and apply it — each keeps a full copy.

Heartbeats and votes

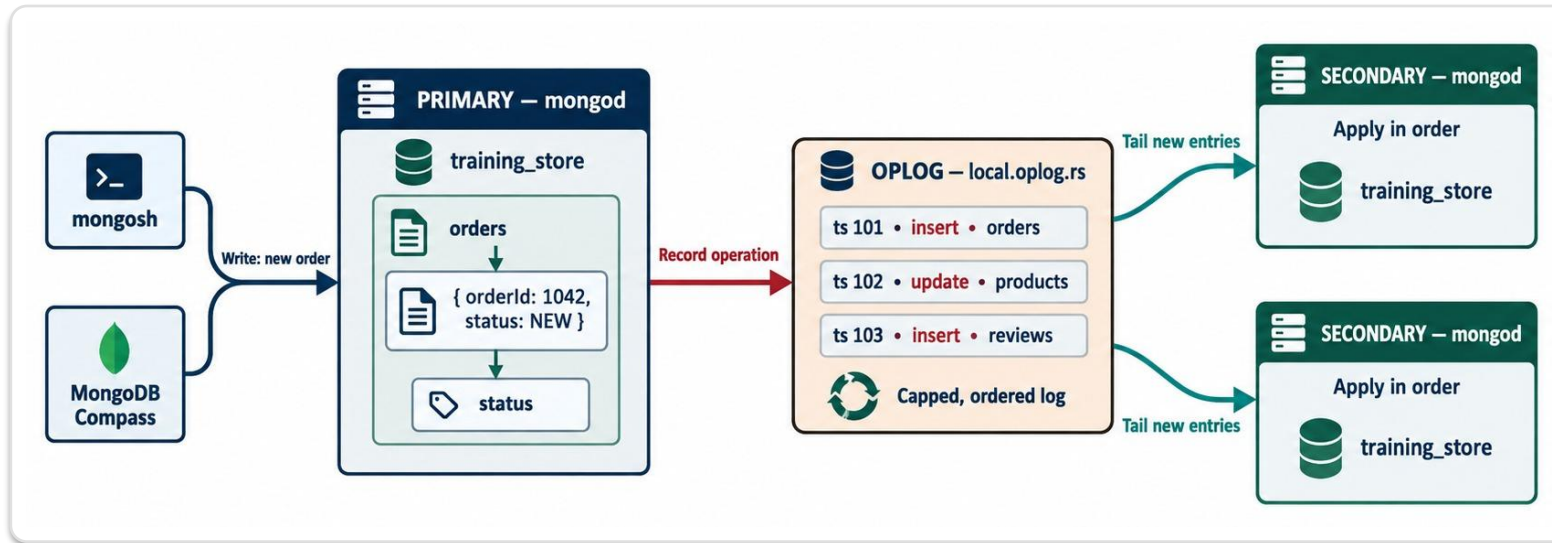
Members check each other every 2 seconds and vote in elections.

THREE MEMBERS The common minimum for automatic failover.

KEY TAKEAWAY A replica set has one writable primary; the secondaries keep full copies, ready to take over.

THE REPLICATION OPLOG

Every change is recorded once — and replayed on every secondary.



Record every change

The primary logs each insert, update and delete in local.oplog.rs.

Secondaries tail it

They pull new entries and apply them in order.

Capped

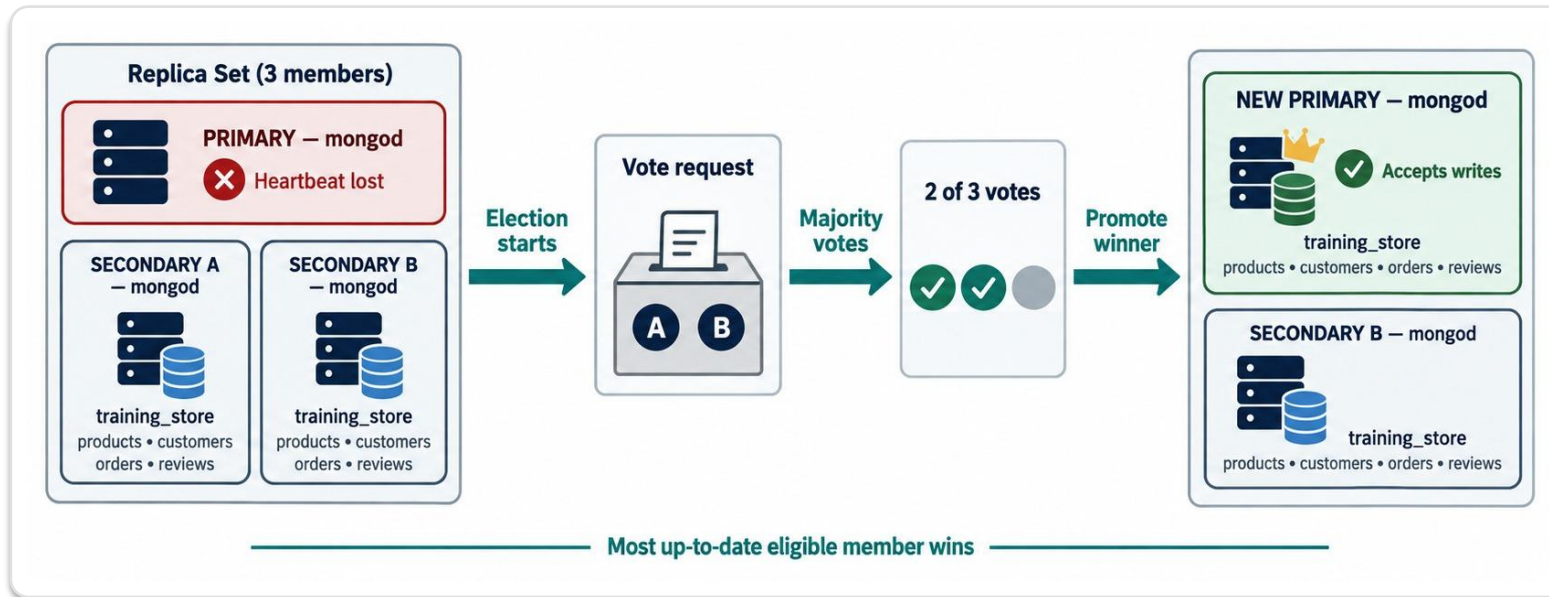
Fixed size: the oldest entries are overwritten — that span is the replication window.

TOO FAR BEHIND A member offline longer than the window needs a full resync.

KEY TAKEAWAY The oplog is the replication stream: secondaries replay it in order to stay in step.

REPLICA-SET ELECTIONS

A candidate needs votes from a majority of voting members.



Trigger

The members stop hearing the primary's heartbeat.

Majority vote

Two of three votes win; the most up-to-date eligible member is chosen.

Why odd numbers are common

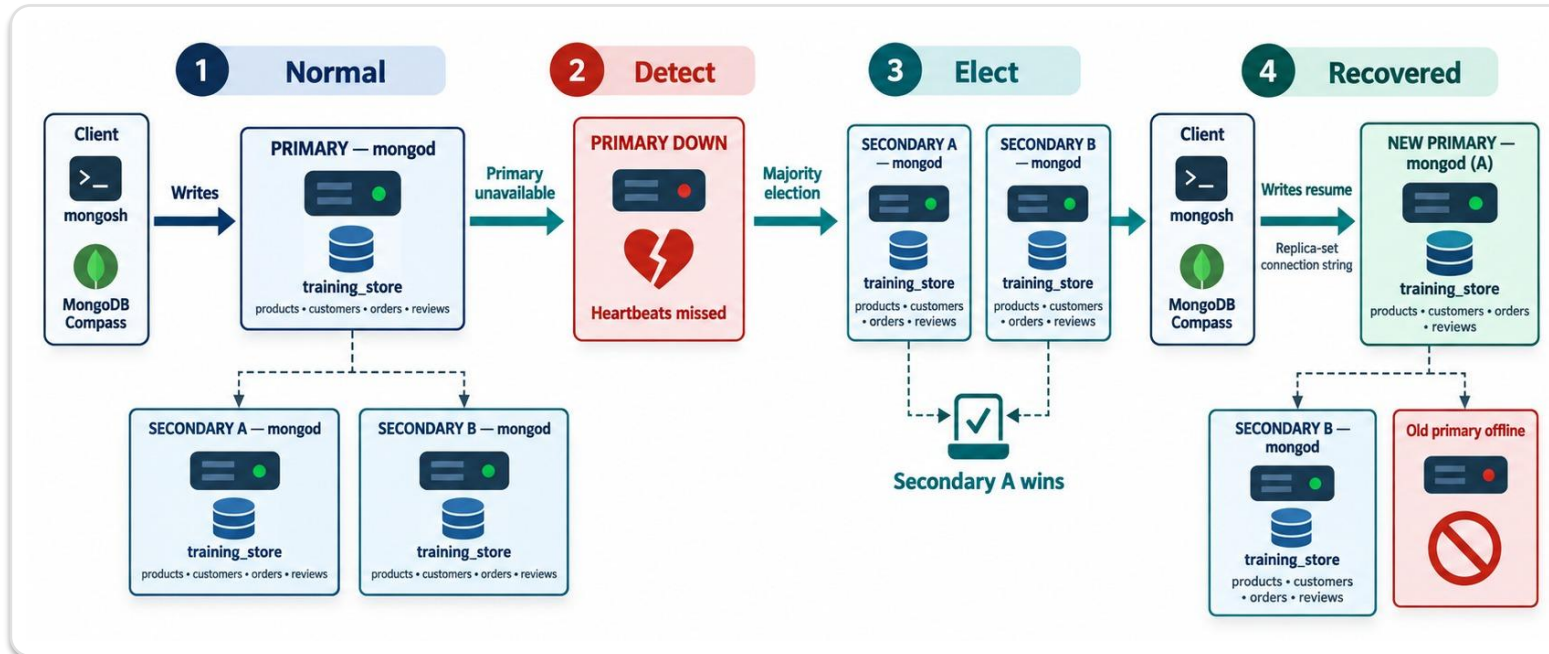
voters	majority	can lose
3	2	1
4	3	1
5	3	2

A FOURTH VOTER Adds cost, not failure tolerance.

KEY TAKEAWAY Elections need a majority of voters — so three or five members, not two or four.

AUTOMATIC FAILOVER

Normal, detect, elect, recovered — without an operator.



1 Normal

Writes go to the primary; the secondaries copy.

2 Detect

Heartbeats are missed — about 10 seconds by default.

3 Elect

Secondary A wins the majority vote and becomes primary.

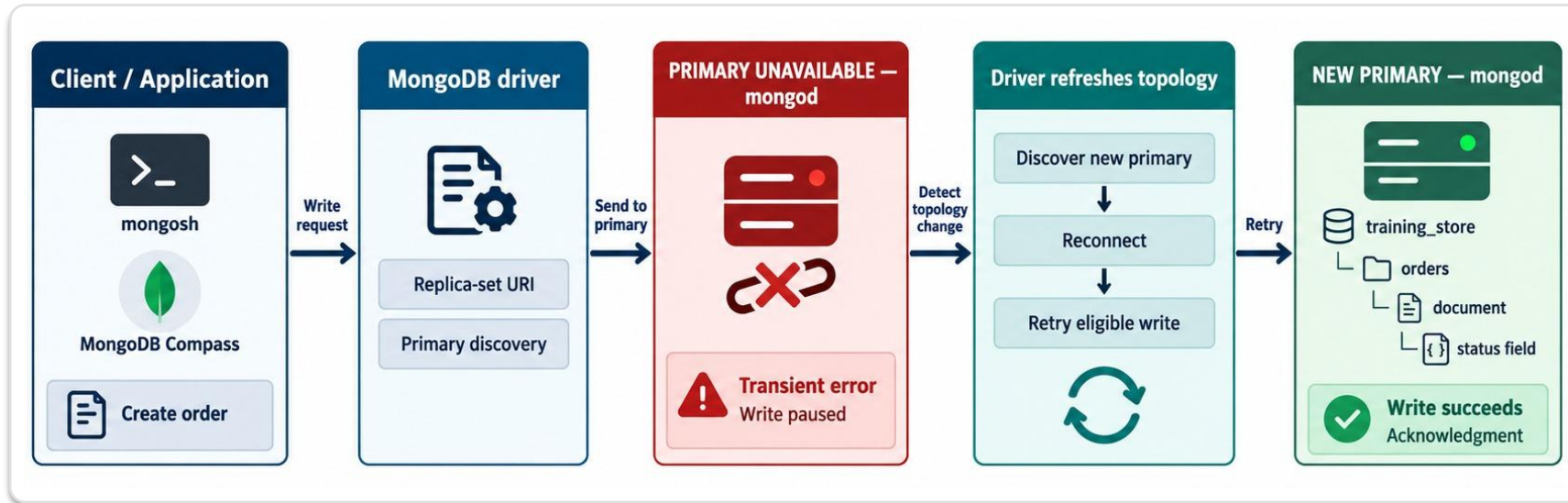
4 Recovered

Writes resume; the old primary rejoins as a secondary.

KEY TAKEAWAY Failover is automatic, but there is a short window with no primary and no writes.

APPLICATION BEHAVIOR DURING FAILOVER

Transient error, topology refresh, retry.



Connect to the whole set

```
mongodb://db1:27017,  
db2:27017,db3:27017/  
training_store  
?replicaSet=trainingRS
```

Transient error

Writes pause during the election;
the driver sees a retryable error.

Retryable writes

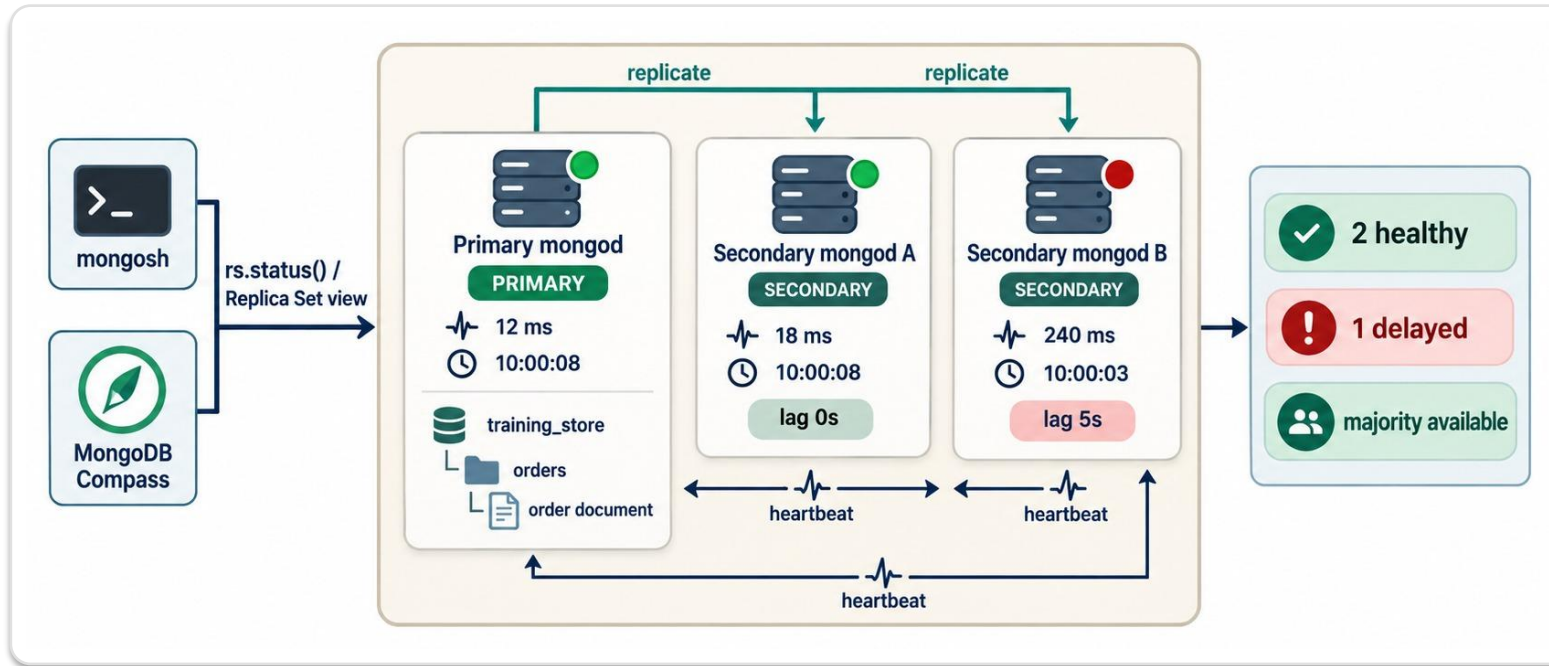
The driver retries an eligible write
once, on the new primary.

DESIGN FOR IT Idempotent
operations and timeouts — never
a pinned host.

KEY TAKEAWAY Use a replica-set connection string and retryable, idempotent operations — failover becomes a pause.

CHECKING REPLICA-SET HEALTH

What rs.status() tells you — and what to watch.



Check health in mongosh

```
rs.status()
rs.conf()
rs.printSecondaryReplicationInfo()
db.hello()
```

Read the states

One PRIMARY, the SECONDARY members, health: 1 on each reachable member.

Watch the lag

Secondary B is 5 s behind — healthy, but worth watching.

LAB 6 Steps 4–7: you'll write this health report on a real replica set.

KEY TAKEAWAY Measure replica-set health — the primary, the majority and the lag — don't guess it.

EXERCISE: COUNT THE VOTES

Decide whether each replica set can still elect a primary.

PRACTICE EXERCISE

10 MIN

SCENARIO

Every member below is a voting, data-bearing member.

- A 3 members, 1 down
- B 3 members, 2 down
- C 4 members, 2 down
- D 5 members split 3 | 2
across two data centers;
the link between them fails

YOUR TASKS

- 1 Work out the majority for each set
- 2 Say whether a primary can exist
- 3 Say what happens to writes
- 4 Explain why D is split 3 | 2

EXPECTED OUTCOME

- ✓ A yes — 2 of 3 still vote
- ✓ B no — 1 of 3: no writes
- ✓ C no — needs 3 of 4
- ✓ D the 3-member side elects
- ✓ Odd voter counts decide cleanly

KEY TAKEAWAY A primary needs a reachable majority of voters — count them before you design.

PART 3: READ AND WRITE GUARANTEES, LAG AND ROLLBACK

Choose where reads go and when a write counts as safe.



SO FAR

- ✓ Writes go to one primary
- ✓ Secondaries replay the oplog
- ✓ A majority elects a new primary

NEXT QUESTION

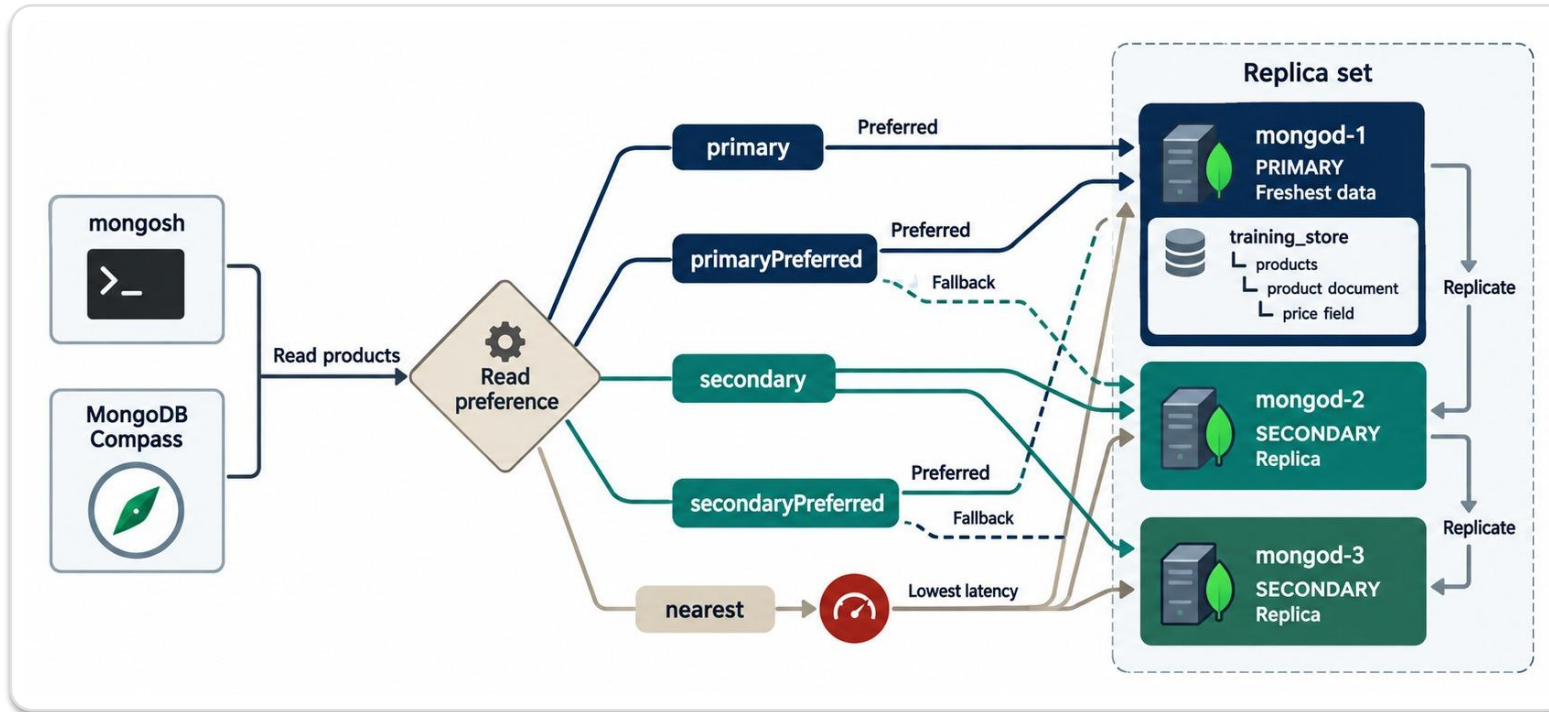
Which member should serve a read — and when is a write really safe?

IN THIS PART

- 1 Read preference
- 2 Write concern
- 3 Read concern
- 4 The three settings in mongosh
- 5 Replication lag
- 6 Rollback
- 7 Exercise: what did the customer see?

READ PREFERENCES

Five modes decide which members may serve a read.



primary (default)

Freshest data — the application reads its own writes.

The Preferred modes

primaryPreferred and secondaryPreferred prefer one side, then fall back.

secondary, nearest

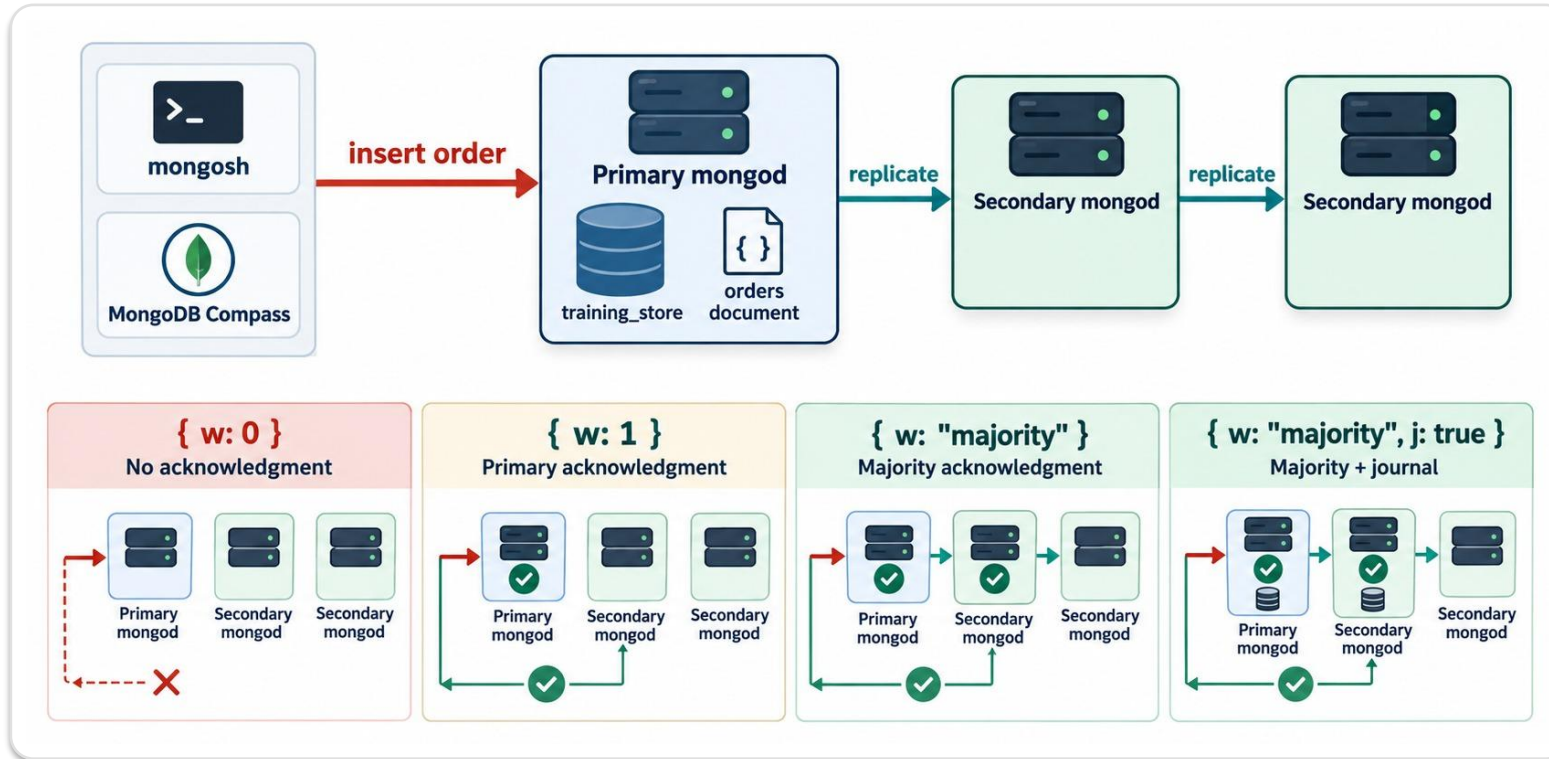
Any eligible secondary, or the lowest-latency member — may be stale.

NOT FREE SCALING Secondary reads can lag and compete with replication.

KEY TAKEAWAY Read from the primary by default; read from secondaries only where stale data is acceptable.

WRITE CONCERN

How many members must have a write before you get an answer.



w: 0

No acknowledgment — errors go unseen.

w: 1

The primary has it — lost if the primary fails before replicating.

w: "majority"

A majority has it — it survives a failover.

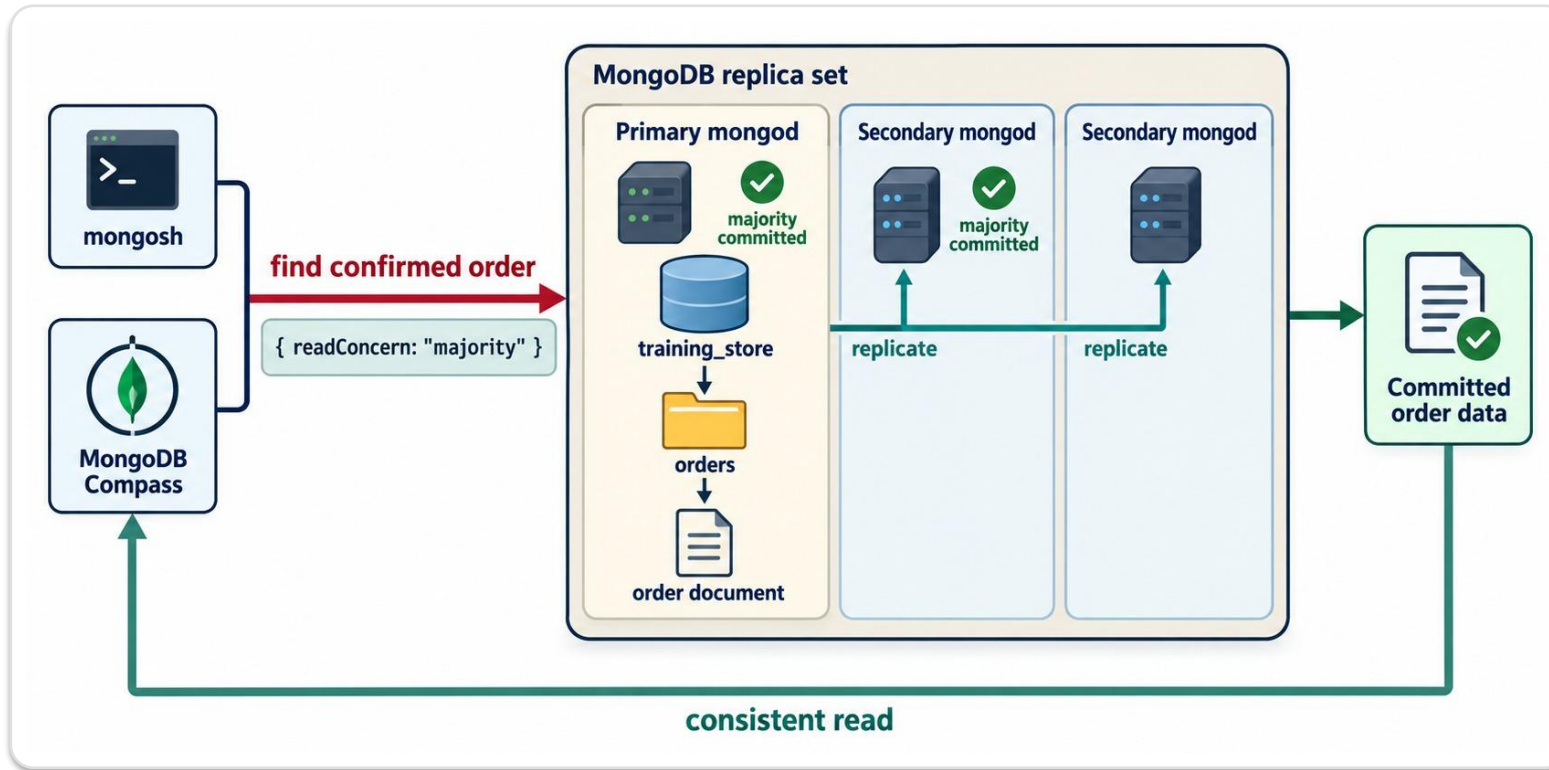
j: true

Also wait for the on-disk journal.

KEY TAKEAWAY Stronger write concern costs some latency and buys durability — match it to the operation.

READ CONCERN

Which data a read may return.



local (default)

The member's newest data — it could still roll back.

majority

Only data a majority has — it will not roll back.

snapshot, linearizable

Transactions, and the strongest single-document reads.

DIFFERENT QUESTION Read preference picks the member; read concern the guarantee.

KEY TAKEAWAY Read preference chooses where a read runs; read concern chooses what it may return.

THE THREE SETTINGS ON TRAINING_STORE

Write concern, read concern and read preference in mongosh.

Write concern: confirm a payment

```
db.orders.updateOne(
  { orderNumber: "06401" },
  { $set: { paymentStatus: "PAID" } },
  { writeConcern: {
    w: "majority",
    wtimeout: 5000 } }
)
```

Write concern

How many members confirm the write before the answer.

Read concern: committed data

```
db.orders.find(
  { paymentStatus: "PAID" }
).readConcern("majority")
```

Read concern

Local data, or only majority-committed data.

Read preference: a report

```
db.getMongo()
  .setReadPref(
    "secondaryPreferred")
db.orders.countDocuments(
  { fulfillmentStatus:
    "SHIPPED" })
```

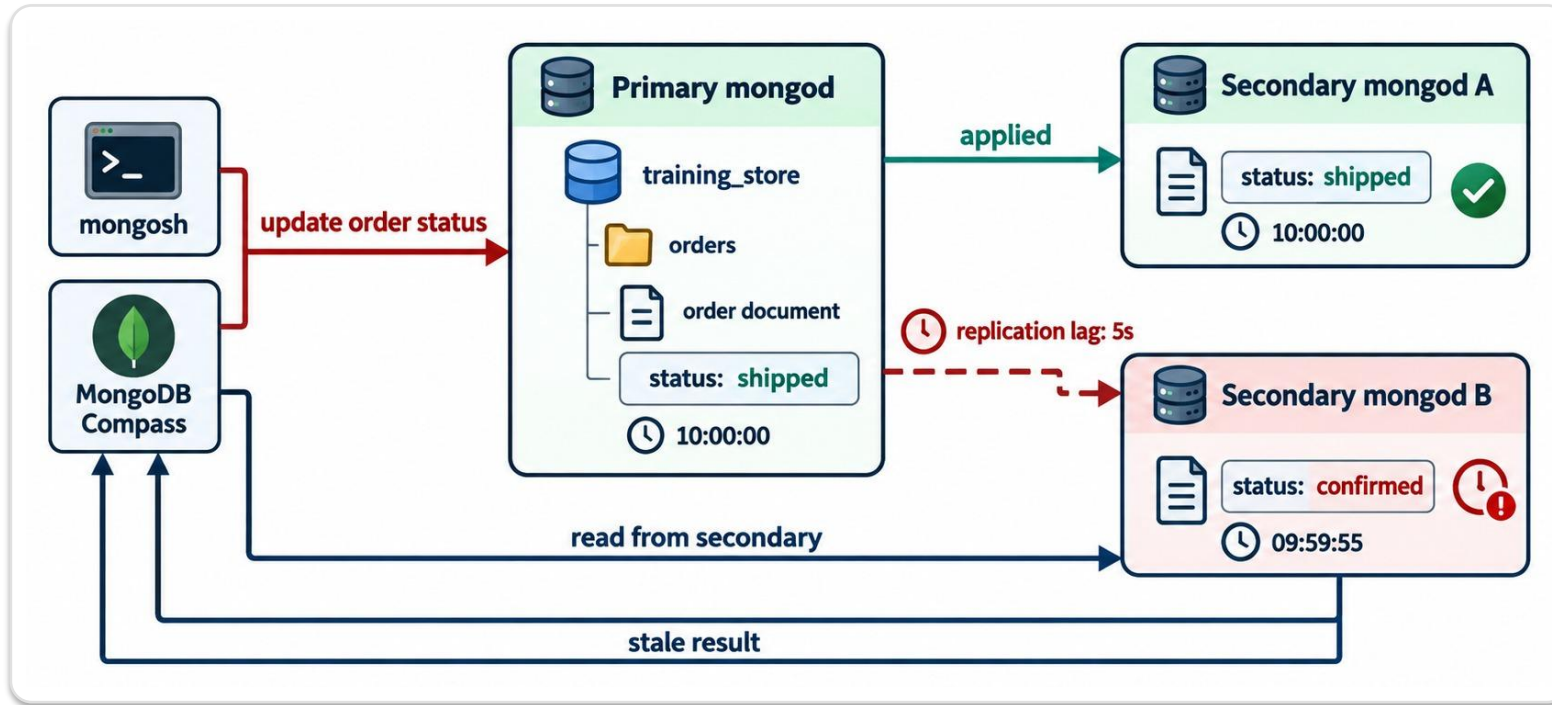
Read preference

Which member serves it — this report may lag.

KEY TAKEAWAY Three different settings: who confirms a write, what a read returns, and where it runs.

REPLICATION LAG

A lagging secondary returns stale data.



What it is

How far a secondary's applied oplog trails the primary — 5 s here.

Stale read

The secondary still says confirmed; the primary says shipped.

Common causes

Slow disk, busy secondary, slow network, write bursts, long operations.

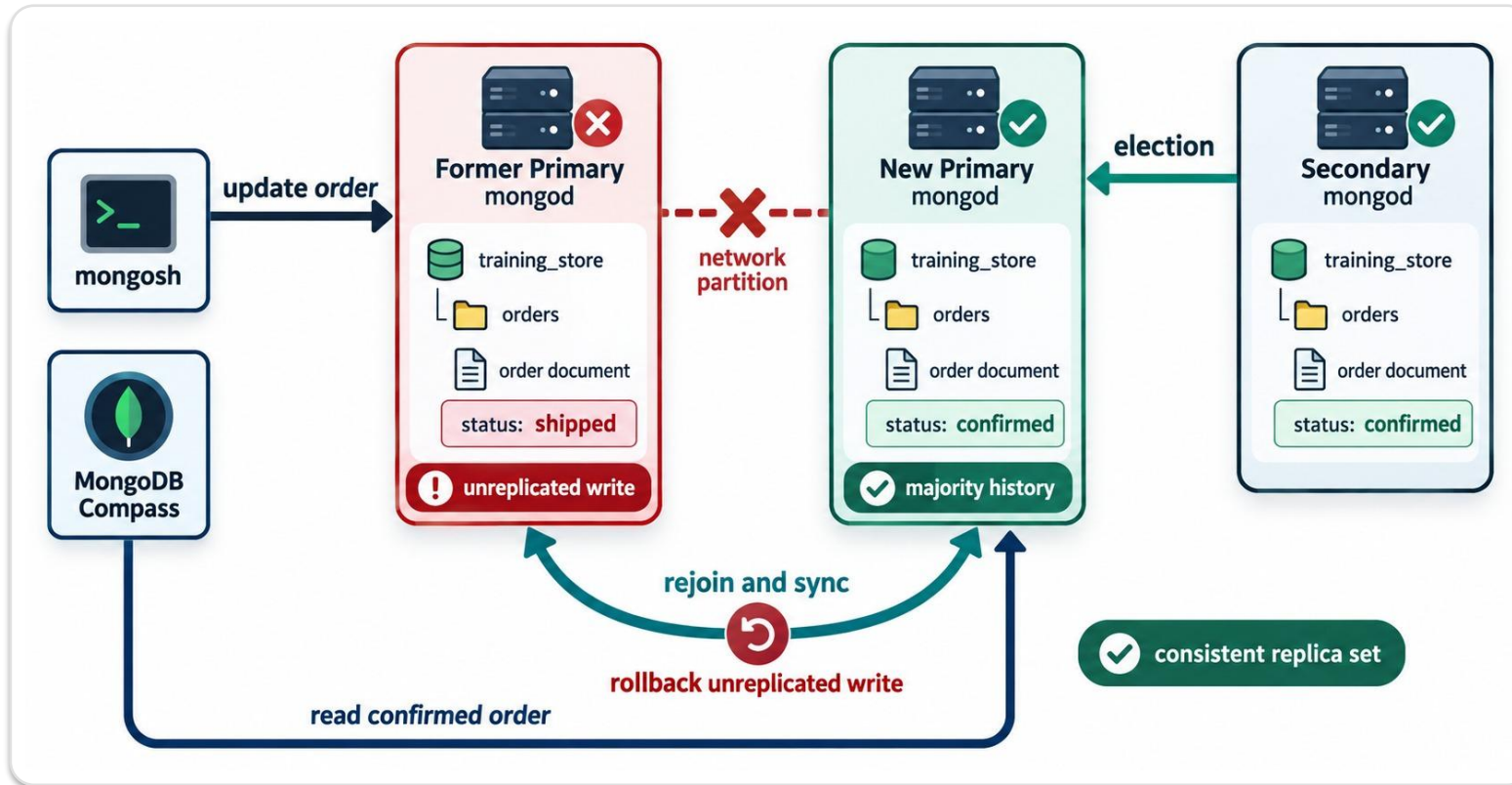
First checks

Member optimes and lag — then the secondary's CPU and disk.

KEY TAKEAWAY Small lag is normal; growing lag means stale reads and slower failover — measure it.

ROLLBACK

A write the majority never saw is undone.



Unreplicated write

The old primary accepted shipped with w: 1, then lost its network.

The majority moves on

The other two members elect a primary that never saw it.

Rollback

The old primary rejoins, undoes the write and saves it to a rollback file.

PREVENTION w: "majority" writes are not rolled back.

KEY TAKEAWAY Only majority-acknowledged writes survive a failover — use them for anything that matters.

EXERCISE: WHAT DID THE CUSTOMER SEE?

Replay a failover during Luis Romero's payment.

PRACTICE EXERCISE

10 MIN

SCENARIO

O6401 is PENDING. The payment service sets paymentStatus to PAID with w: 1. The status page reads with secondaryPreferred.

```
10:00:00 PAID written on primary
10:00:00 acknowledged (w: 1)
10:00:01 primary partitioned
10:00:12 secondary elected
10:00:30 old primary rejoins
```

YOUR TASKS

- 1 State paymentStatus on the new primary
- 2 Say what happens to the PAID write
- 3 Say what the status page showed
- 4 Pick the settings you would change

EXPECTED OUTCOME

- ✓ PENDING — it never got PAID
- ✓ Rolled back, saved to a file
- ✓ PENDING: stale, then wrong
- ✓ Payment: w: "majority"
- ✓ Status page: primary reads

KEY TAKEAWAY w: 1 can lose an acknowledged write; majority writes plus primary reads keep the customer's view true.

PART 4: SHARDED CLUSTERS AND SHARD KEYS

Split orders across servers — and choose the key that splits them.



SO FAR

- ✓ Replica sets survive a server failure
- ✓ Settings trade freshness for load and latency
- ✓ Majority writes survive failover

NEXT QUESTION

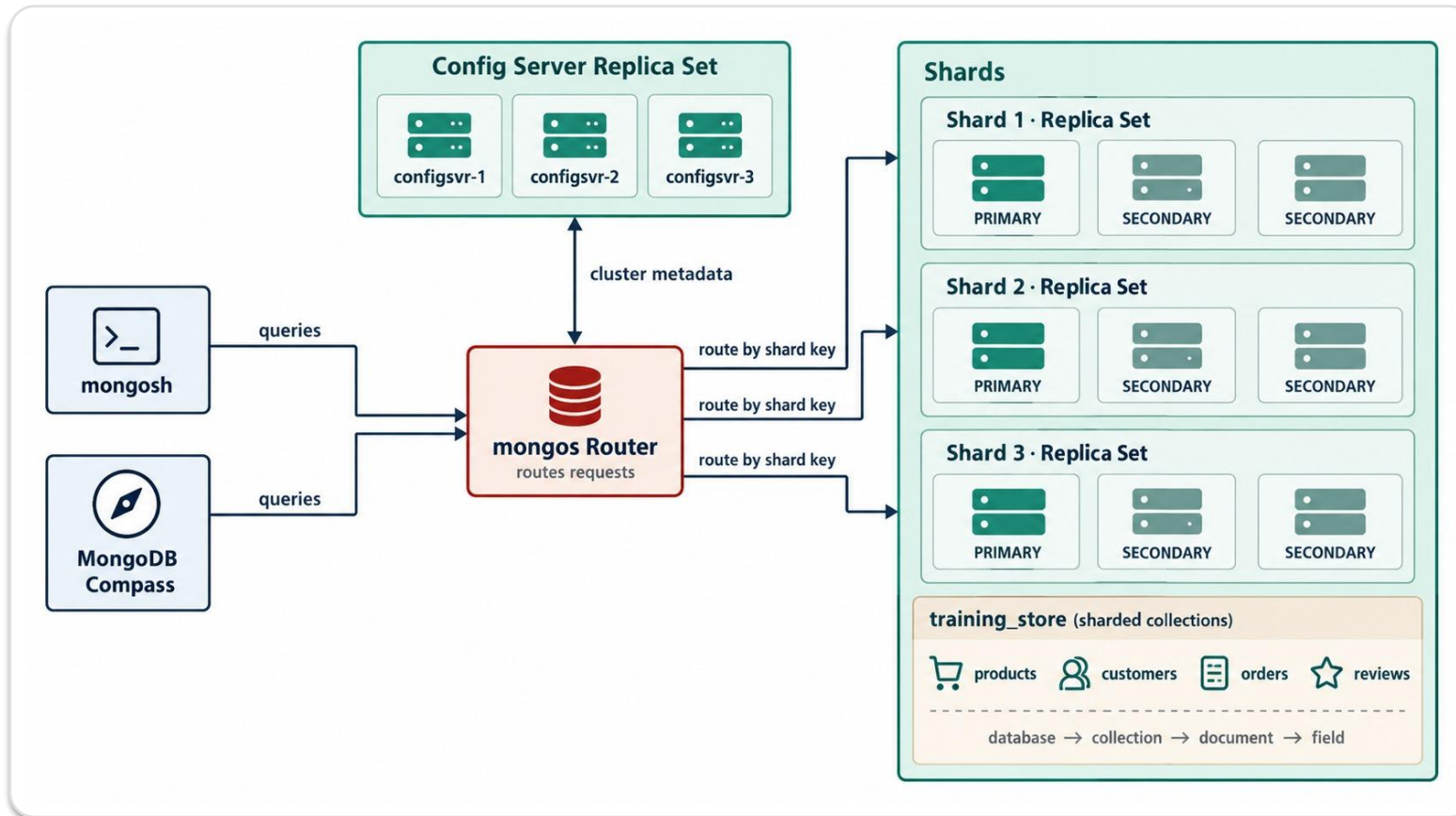
How is a collection split across shards — and which field should split it?

IN THIS PART

- 1 Sharded-cluster components
- 2 Shards as replica sets
- 3 Cardinality and frequency
- 4 Monotonic keys
- 5 Ranged sharding
- 6 Hashed and zoned sharding
- 7 Candidate keys for orders

SHARDED-CLUSTER COMPONENTS

mongos routers, a config server replica set and replica-set shards.



mongos

The router applications connect to; it stores no collection data.

Config server replica set

Holds the metadata: which ranges live on which shard.

Shards

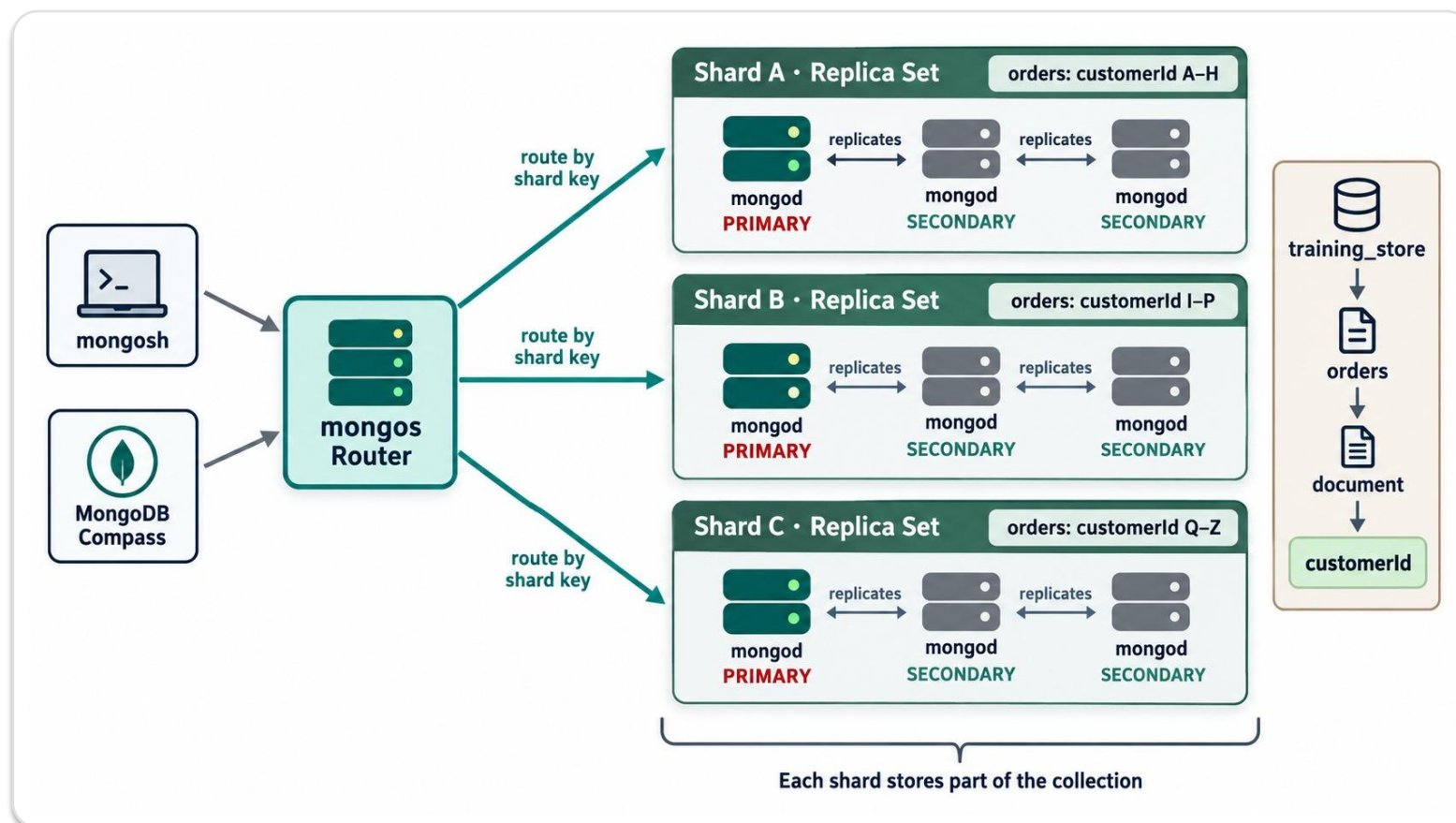
Each is a replica set holding part of the sharded data.

ONLY SHARDED COLLECTIONS SPLIT The rest stay on one shard.

KEY TAKEAWAY Applications talk to mongos; config servers hold the map; shards hold the data.

SHARDS

Each shard is a replica set holding part of the collection.



Part of the collection

Each shard owns its own ranges of orders — here by customerId.

A replica set per shard

A primary and two secondaries, with its own elections.

Illustrative labels

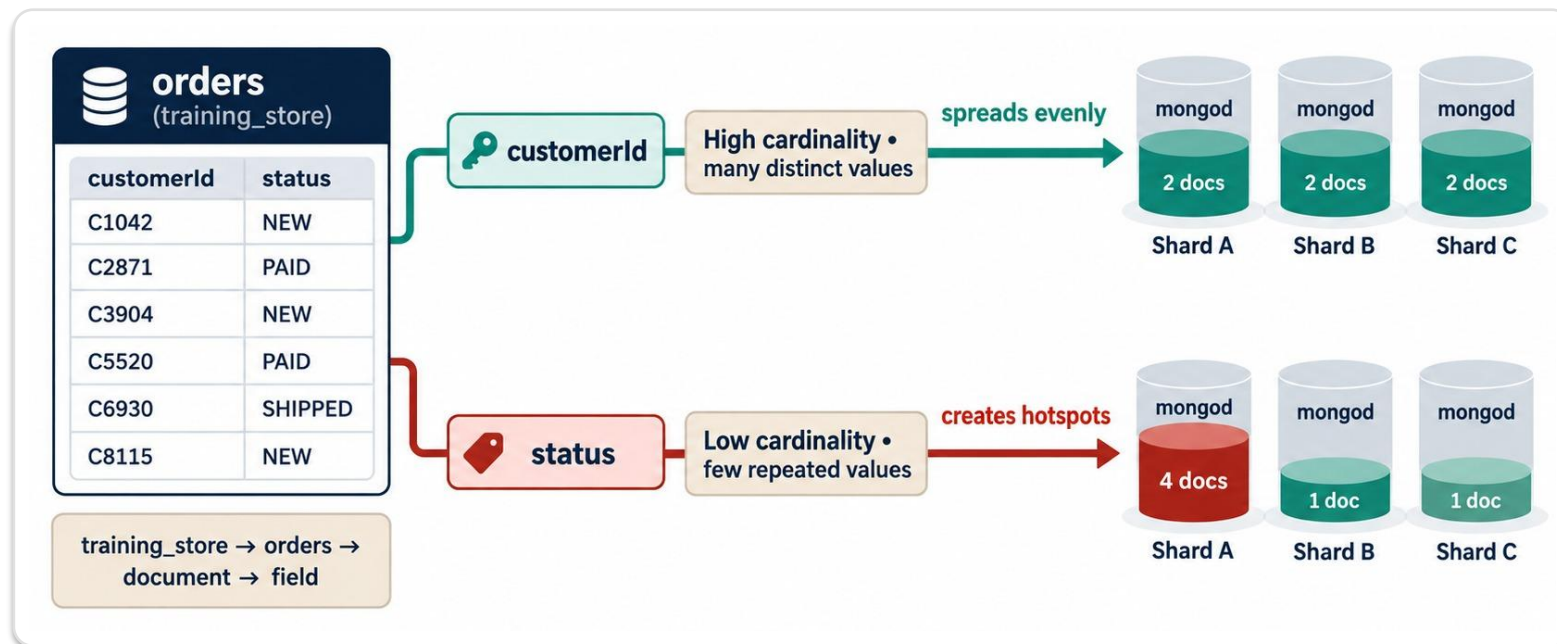
A-H, I-P, Q-Z stand in for ranges; our customerId is an ObjectId.

NEVER A SINGLE MONGOD An unreplicated shard is a single point of failure.

KEY TAKEAWAY Sharding distributes the data between shards; replication keeps each shard available.

CARDINALITY AND FREQUENCY

Many distinct values spread evenly; a few values create hotspots.



customerId: high

Many distinct values — many split points, an even spread.

status: low

A handful of values — each one is a bucket that can't split.

Frequency matters too

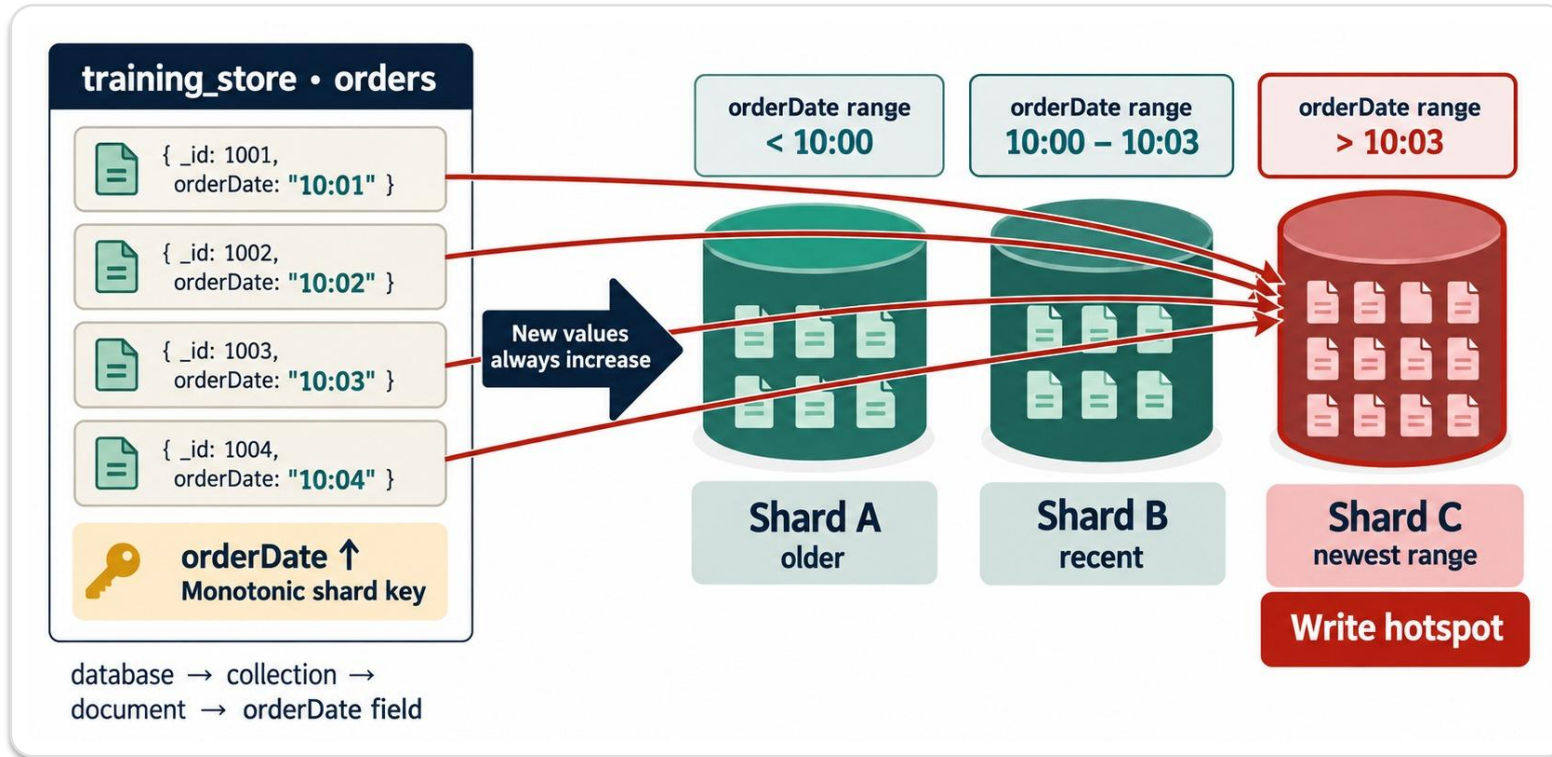
One huge customer can still make one range hot.

TRAINING_STORE fulfillmentStatus has only four values.

KEY TAKEAWAY Choose a shard key with many values that occur evenly — never a status field.

MONOTONIC SHARD KEYS

Always-increasing values pile into the newest range.



Always increasing

createdAt, sequential order numbers, ObjectId-based _id.

Ranged key → hotspot

Every new order lands in the top range, on one shard.

Fixes

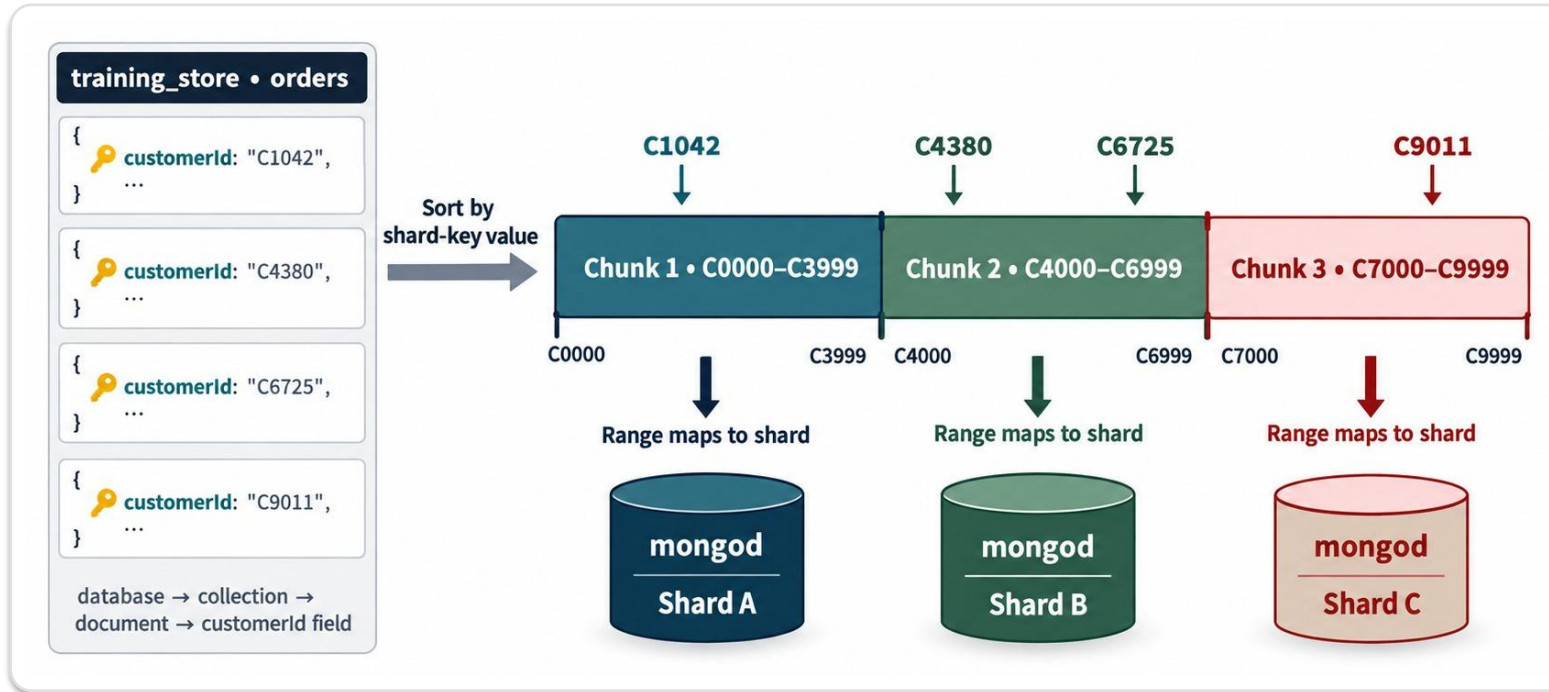
Hash the value, or lead with a high-cardinality field such as customerId.

DIAGRAM FIELD orderDate stands for our createdAt.

KEY TAKEAWAY A monotonic ranged key sends every new write to one shard — hash it or lead with another field.

RANGED SHARDING

Sorted shard-key values cut into contiguous ranges.



Contiguous ranges

Sorted values are cut into ranges; each range lives on one shard.

Good for ranges

One customer's orders, or a date span, stay on few shards.

Risk

A monotonic key sends all new writes to the last range.

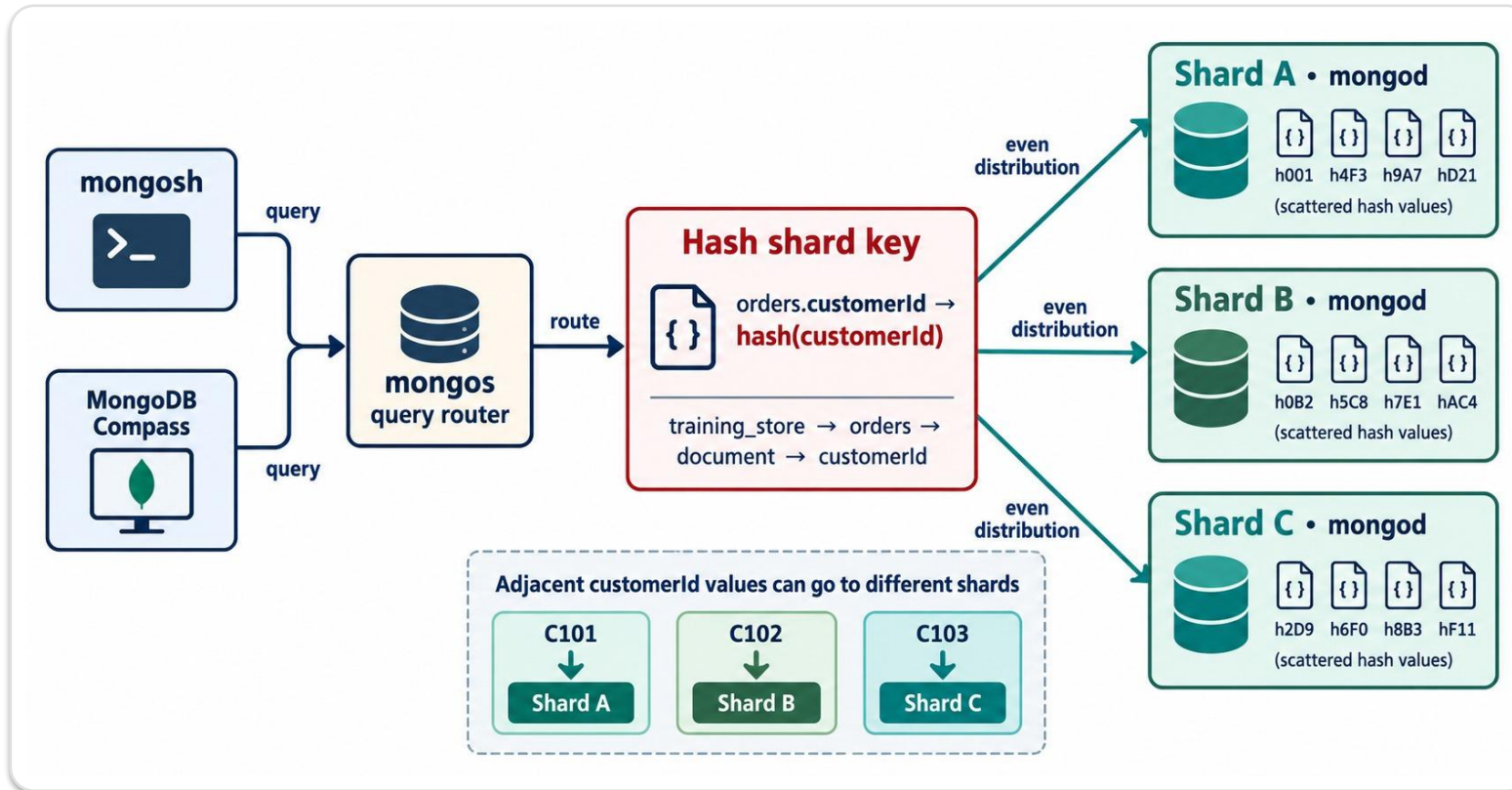
A ranged compound key

```
{ customerId: 1, createdAt: 1 }
```

KEY TAKEAWAY Ranged sharding keeps neighbours together — great for range queries, risky for rising keys.

HASHED SHARDING

Hash the value, then distribute the hash ranges.



Hash first

MongoDB hashes `customerId` and ranges the hash values.

Even spread

Neighbours C101, C102 and C103 land on different shards.

Trade-off

Equality finds stay targeted; range queries on the field scatter.

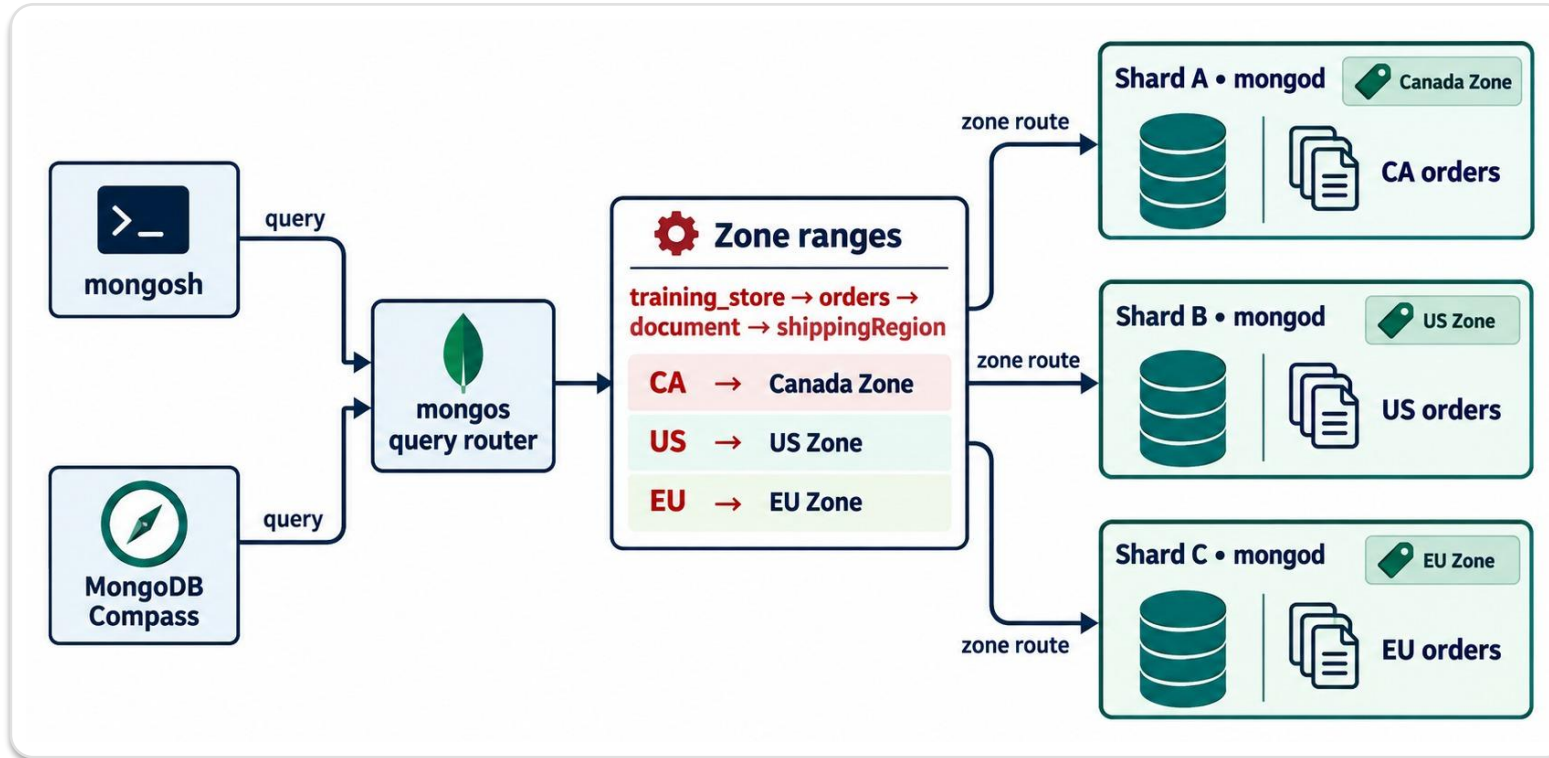
A hashed key

```
{ customerId: "hashed" }
```

KEY TAKEAWAY Hashed sharding spreads writes evenly — and gives up efficient range queries on that field.

ZONED SHARDING

Zone ranges pin chosen data to chosen shards.



Pin ranges to shards

Canadian orders stay on the Canada-zone shard.

Why

Data residency, and keeping data near its users.

Needs a key prefix

We'd add a region field and lead the shard key with it.

WATCH THE SKEW A popular zone means hot shards.

KEY TAKEAWAY Zones place data deliberately — for residency or locality — but can concentrate load.

COMPARING CANDIDATE SHARD KEYS FOR ORDERS

Six candidates for training_store.orders — five on real fields, one what-if.

Candidate	Strength	Concern
{ fulfillmentStatus: 1 }	Status queries	4 values: low cardinality
{ createdAt: 1 }	Date-range queries	Monotonic: write hotspot
{ customerId: 1 }	Customer history	A huge customer concentrates
{ customerId: "hashed" }	Even write spread	Ranges scatter
{ customerId: 1, createdAt: 1 }	History + date range	Depends on customer skew
{ region: 1, customerId: 1 }	Regional placement	Needs a new region field

Our pick to test

{ customerId: 1, createdAt: 1 } — it matches Module 6's history query.

Validate it

Test with production-like data and traffic before you choose.

NO UNIVERSAL KEY Choose from the workload.

KEY TAKEAWAY No shard key is universally right — score each candidate against the real workload.

SHARDING A COLLECTION — INSTRUCTOR DEMONSTRATION

Run through mongos, on a disposable cluster only.

Ranged compound key

```
// disposable lab cluster only
db.orders.createIndex(
  { customerId: 1, createdAt: 1 })
sh.shardCollection(
  "training_store.orders",
  { customerId: 1, createdAt: 1 })
```

Or a hashed key

```
sh.shardCollection(
  "training_store.orders",
  { customerId: "hashed" })
```

Inspect the result

```
sh.status()
db.orders
  .getShardDistribution()
```

Through mongos

Sharding commands run on a mongos connection.

Index first

The shard key needs a supporting index.

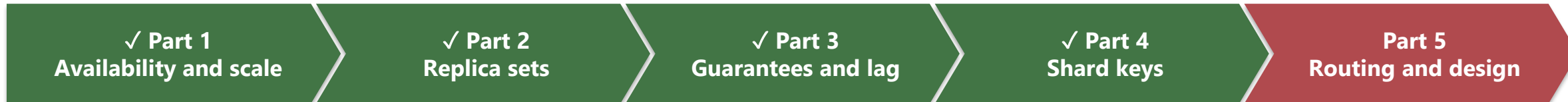
Disposable only

Never shard the shared class cluster.

KEY TAKEAWAY Shard with a supporting index, through mongos, on a disposable cluster — then inspect the result.

PART 5: DISTRIBUTION, ROUTING AND DESIGN

How ranges move, how queries find them — and when to shard at all.



SO FAR

- ✓ mongos, config servers and replica-set shards
- ✓ A good key: many values, even use, not monotonic
- ✓ Ranged, hashed and zoned sharding

NEXT QUESTION

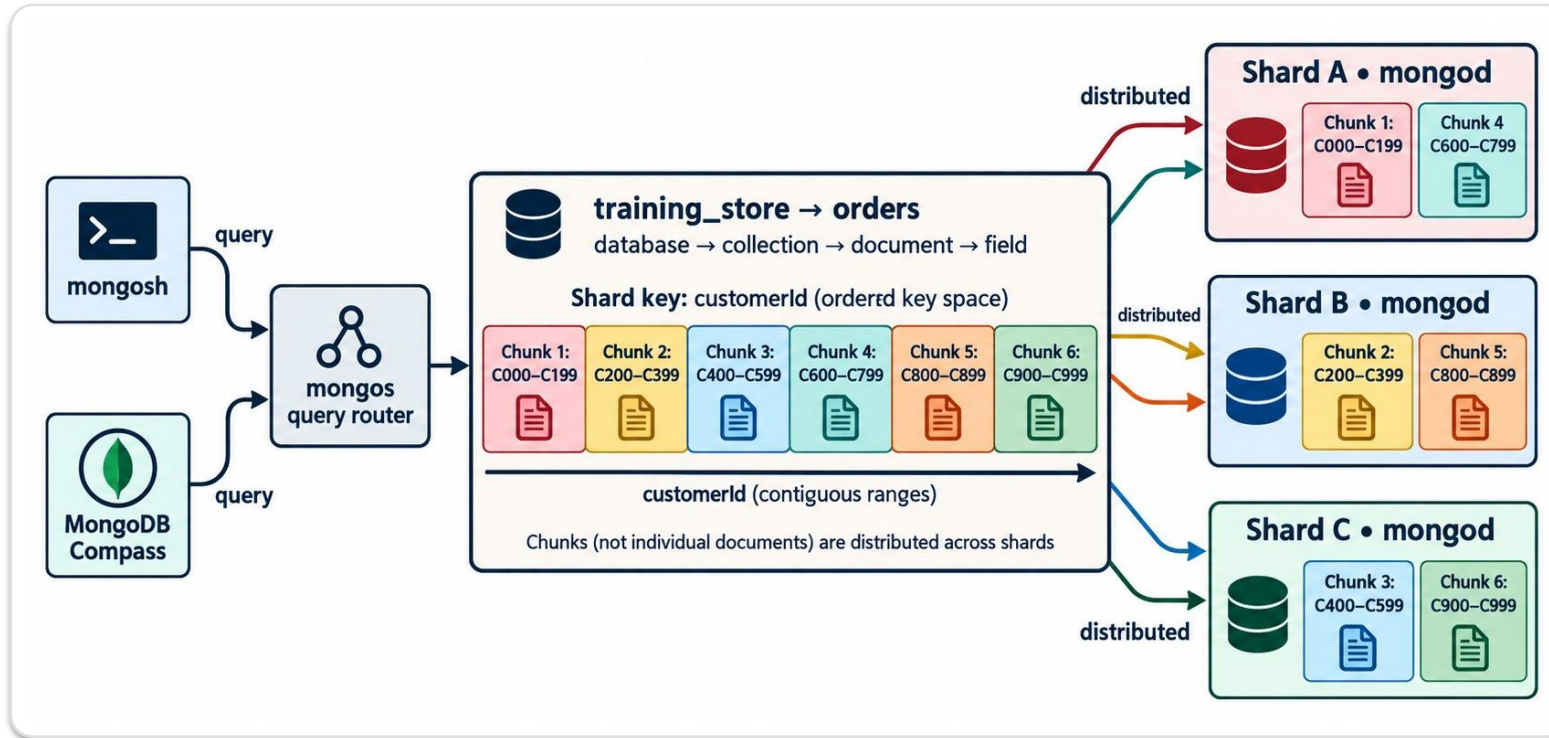
How do ranges stay balanced and queries find them — and when should we shard?

IN THIS PART

- 1 Chunks
- 2 The balancer
- 3 Targeted queries
- 4 Targeted versus scatter-gather
- 5 Replication and sharding together
- 6 The deployment decision
- 7 Misconceptions and anti-patterns

CHUNKS AND DATA DISTRIBUTION

The key space is cut into ranges; ranges are spread over shards.



Chunks are ranges

Each non-overlapping range of the key lives on one shard.

Not documents

MongoDB moves whole ranges, never single documents.

Metadata

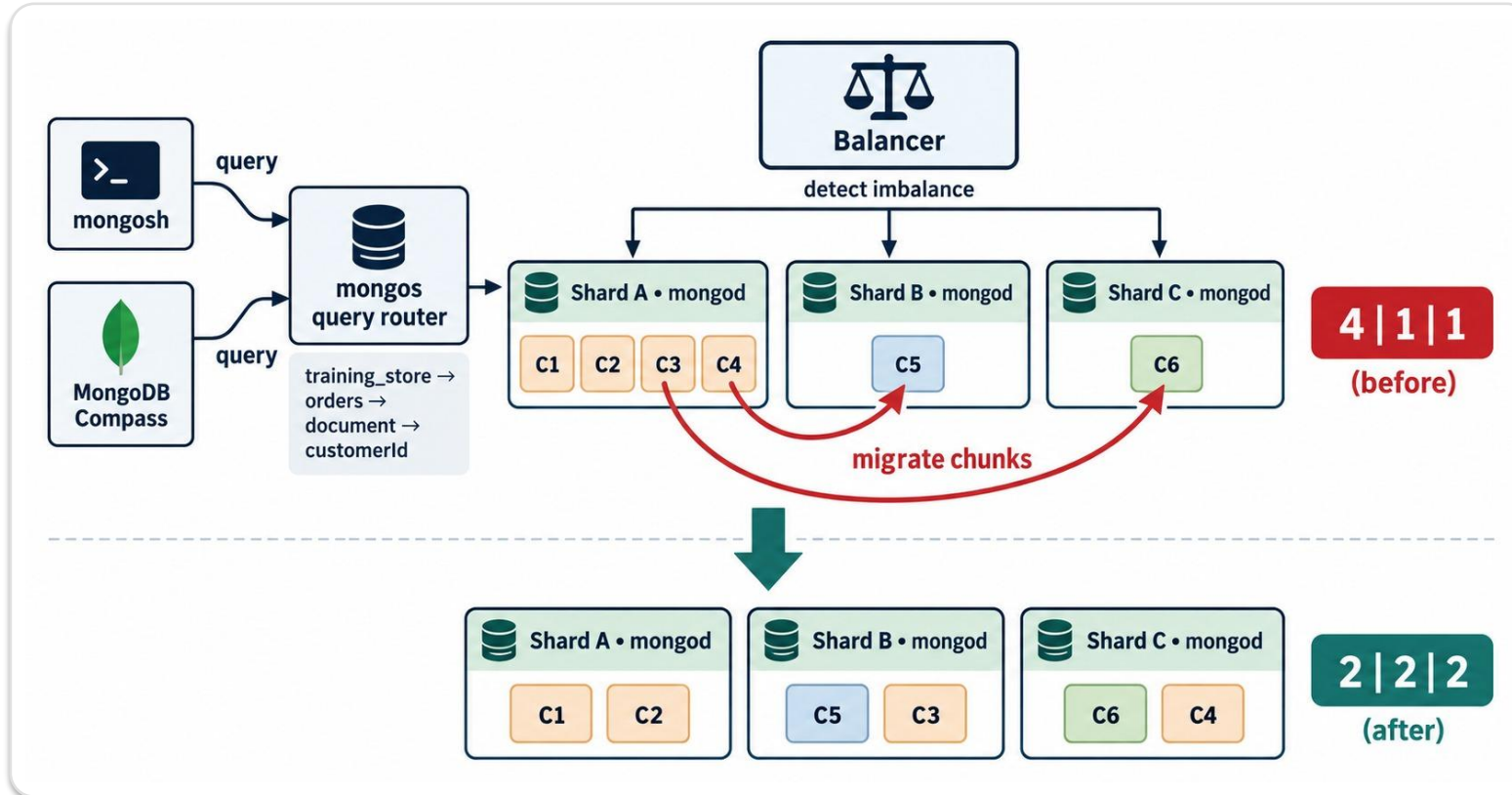
The config servers record which shard owns each range.

TRAINING DATA Tiny data sets often sit in one range.

KEY TAKEAWAY Data is placed and moved range by range — the chunk map is what mongos routes with.

THE BALANCER

Detect imbalance, migrate ranges, even out the shards.



Detect imbalance

Shard A holds four chunks; Shards B and C hold one each.

Migrate ranges

C3 moves to Shard B, C4 to Shard C — two chunks each.

Costs resources

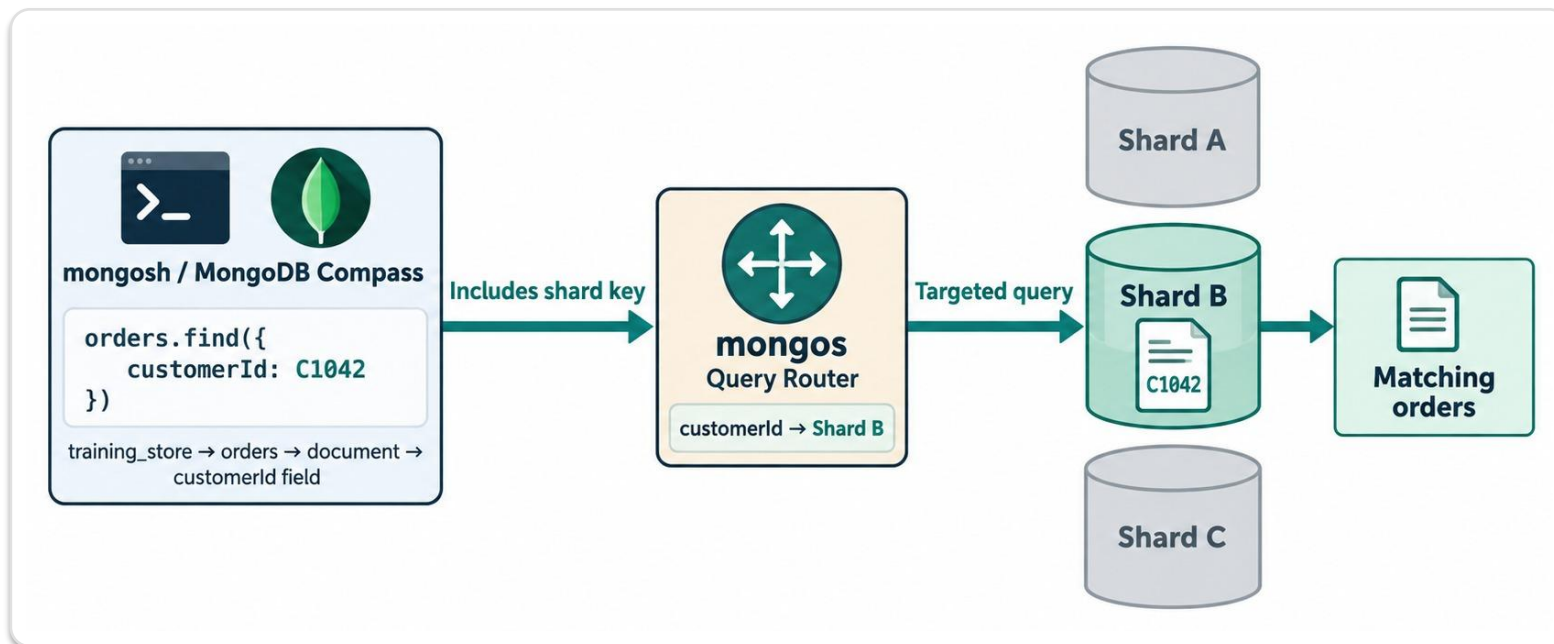
Migrations use network, disk and CPU — monitor them.

MODERN VERSIONS Balance by data size, not chunk count.

KEY TAKEAWAY The balancer evens out the shards automatically — but migrations have a cost, so watch them.

TARGETED QUERIES

A query that includes the shard key goes to one shard.



Includes the shard key

`find({ customerId: ... })` names the key.

mongos routes

The chunk map says Shard B — only Shard B works.

The fastest shape

Shards A and C stay free for other queries.

VALUE SHOWN C1042 is illustrative; ours is an ObjectId.

KEY TAKEAWAY Put the shard key in your hot-path queries so mongos can target one shard.

TARGETED OR SCATTER-GATHER ON TRAINING_STORE?

Assume orders is sharded on { customerId: 1, createdAt: 1 }.

✓ Targeted: shard-key prefix

```
const c = db.customers.findOne(
  { customerNumber: "C101" })
db.orders
  .find({ customerId: c._id })
  .sort({ createdAt: -1 })
```

Targeted

mongos sends it to the shard owning that customer's range.

✗ Scatter-gather: no shard key

```
db.orders.find(
  { fulfillmentStatus:
    "PROCESSING" })
db.orders.find(
  { _id: someOrderId })
```

Scatter-gather

Every shard runs it; mongos merges the results.

Check it with explain

```
db.orders
  .find({ customerId: c._id })
  .explain()
// winningPlan lists
// the shards used
```

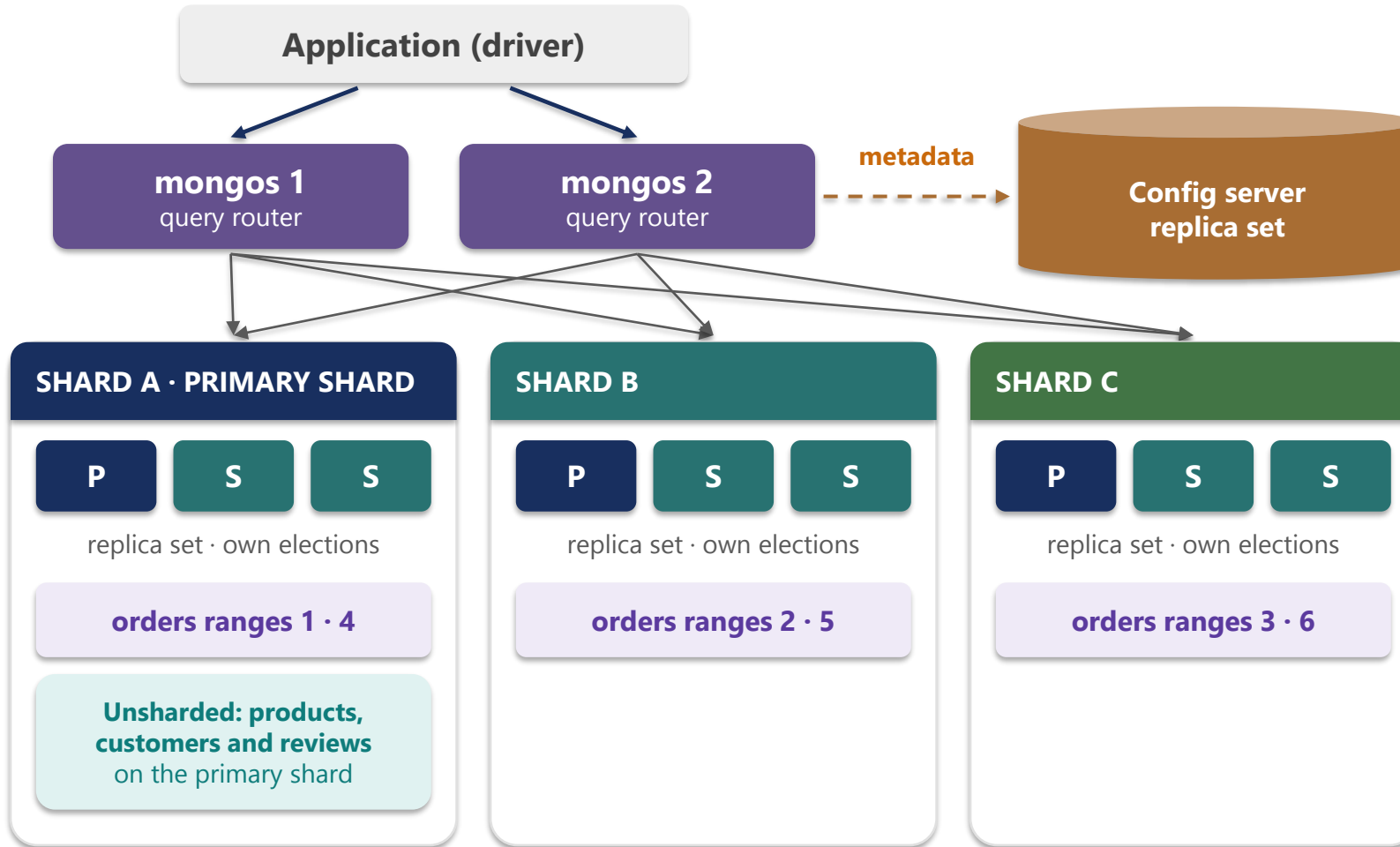
Not always wrong

Fine for rare reports — costly in the hot path.

KEY TAKEAWAY Queries with the shard-key prefix are targeted; the rest ask every shard — keep those rare.

REPLICATION AND SHARDING TOGETHER

A production-style cluster: orders sharded, every shard a replica set.



Sharding splits orders

Its ranges are spread over three shards; mongos routes each request by shard key.

Replication protects each shard

Every shard is its own replica set — P is the primary, S a secondary.

If a Shard B server fails

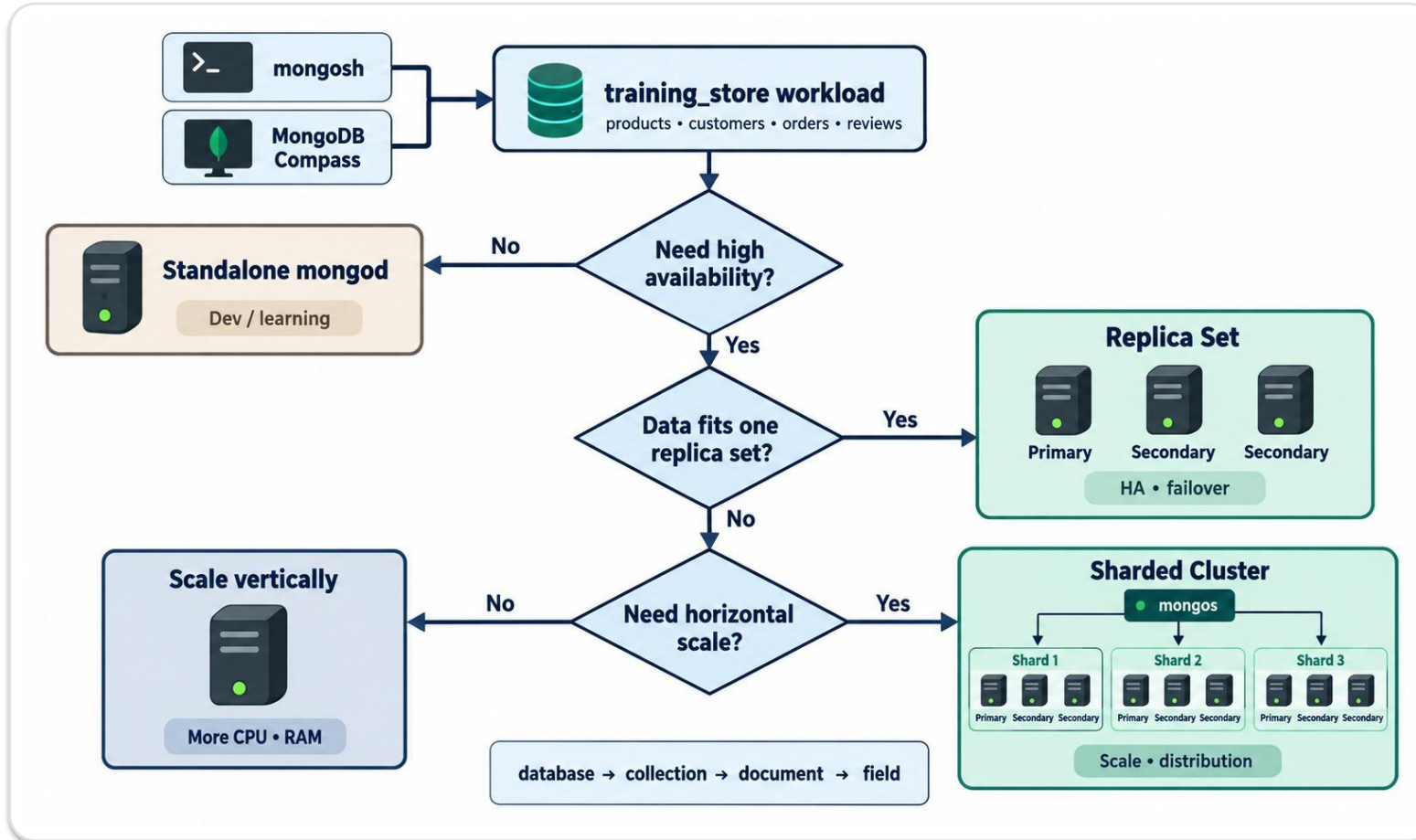
Shard B elects a new primary; Shards A and C keep serving.

PRIMARY SHARD Unsharded collections live on one shard, not on all.

KEY TAKEAWAY Sharding distributes the data; replication protects every shard — production clusters need both.

DEPLOYMENT DECISION FRAMEWORK

Standalone, replica set, scale up — or shard.



Development or learning

A standalone mongod is enough.

Production

Start with a replica set of at least three members.

Outgrown one replica set?

Scale up first; shard when one set can't hold the data or writes.

SHARD WITH EVIDENCE Fix schema and indexes first.

KEY TAKEAWAY Replicate every production deployment; shard only when measurements show you must.

MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs that lead first distributed designs into trouble.

MISCONCEPTION

REALITY

A replica set spreads writes over its members

All writes go to the primary; the secondaries copy them

Secondary reads always make apps faster

They may be stale and compete with replication

Sharding speeds up every query

Queries without the shard key ask every shard

Replication is a backup

A deleteMany is replicated too — keep real backups

The shard key is just another index

It decides placement and routing — and is hard to change

ANTI-PATTERNS TO AVOID

- ✗ Connecting to one host, not the set
- ✗ Two or four voters instead of three or five
- ✗ Sharding on fulfillmentStatus
- ✗ A ranged key on createdAt
- ✗ Sharding before fixing indexes

REMEMBER Replicate to stay online, shard to grow, back up to recover.

KEY TAKEAWAY Most distributed-design problems come from mixing up availability, scale and recovery.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What is the main purpose of replication? Of sharding?

2 What does the oplog do?

3 What starts an election, and why is a majority needed?

4 What is replication lag, and why does it matter?

5 Read preference, read concern, write concern: what does each control?

6 What does mongos do? What do config servers store?

7 Why is fulfillmentStatus a poor shard key?

8 Why can a ranged key on createdAt create a hotspot?

9 Targeted versus scatter-gather: what is the difference?

10 Is replication a replacement for backups?

EXERCISE 7.1 — REPLICATION OR SHARDING?

Module 7 checkpoint A: match each requirement to its mechanism.

OFFICIAL CHECKPOINT A

10 MIN

labs/day-03/exercises/exercise-7.1-replication-or-sharding.md

SCENARIO

Write Replication, Sharding, Both or Neither yet for each requirement.

- 1 Survive one server failure
- 2 Increase total data capacity
- 3 Elect a replacement primary
- 4 Distribute write load
- 5 A local development database
- 6 Production availability + scale

YOUR TASKS

- 1 Classify requirements 1–3
- 2 Classify requirements 4–6
- 3 Explain why secondaries add no capacity
- 4 Compare with Lab 6: last week's delete

SOLUTION (AFTER DEBRIEF)

- ✓ 1 Replication · 2 Sharding
- ✓ 3 Replication · 4 Sharding
- ✓ 5 Neither yet: standalone
- ✓ 6 Both: sharded replica sets
- ✓ Same data on each member

KEY TAKEAWAY Copies give availability, shards give capacity — and development needs neither yet.

EXERCISE 7.2 — SELECT READ AND WRITE SETTINGS

Module 7 checkpoint B: choose settings per business operation.

OFFICIAL CHECKPOINT B

20 MIN

labs/day-03/exercises/exercise-7.2-select-read-and-write-settings.md

SCENARIO

For each operation choose a read preference, a read concern and a write concern.

Payment confirmation
Financial reconciliation
Product-catalog browsing
Customer order status
Operational reporting
Noncritical analytics

YOUR TASKS

- 1 Set the two money paths
- 2 State what you refuse to trade away
- 3 Set catalog, status and reporting
- 4 One sentence on staleness for each

SOLUTION (AFTER DEBRIEF)

- ✓ Payment: primary + majority
- ✓ Reconciliation: same bias
- ✓ Catalog: primaryPreferred ok
- ✓ Status: primary after payment
- ✓ Analytics: secondaryPreferred

KEY TAKEAWAY Money paths get primary reads and majority writes; staleness is accepted only where it's harmless.

EXERCISE 7.3 — EVALUATE SHARD-KEY CANDIDATES

Module 7 checkpoint C: score keys for `training_store.orders`.

OFFICIAL CHECKPOINT C

20 MIN

labs/day-03/exercises/exercise-7.3-evaluate-shard-key-candidates.md

SCENARIO

Score each candidate 1–5 on cardinality, distribution, write scale, targeting, hotspot risk, locality and growth. `tenantId` is a what-if: `training_store` has none.

```
{ paymentStatus: 1 }
{ createdAt: 1 }
{ customerId: 1 }
{ customerId: "hashed" }
{ tenantId: 1, orderNumber: 1 }
{ customerId: 1, createdAt: 1 }
```

YOUR TASKS

- 1 Score all six candidates
- 2 Reject the low-cardinality key
- 3 Flag the monotonic key
- 4 Recommend one; name two risks

SOLUTION (AFTER DEBRIEF)

- ✓ `paymentStatus`: rejected
- ✓ `createdAt`: write hotspot
- ✓ Hashed: even, no ranges
- ✓ Pick `customerId` + `createdAt`
- ✓ Risks: big customers, reports

KEY TAKEAWAY A defensible shard key matches the hot query — and still names the risks you'll monitor.

EXERCISE 7.4 — TARGETED OR SCATTER-GATHER?

Module 7 checkpoint D: classify five queries by their routing.

OFFICIAL CHECKPOINT D

15 MIN

labs/day-03/exercises/exercise-7.4-targeted-or-scatter-gather.md

SCENARIO

Shard key: { customerId: 1, createdAt: 1 }. c is customer C101.

```
A { customerId: c._id }
B { customerId: c._id,
  createdAt: { $gte: start } }
C { paymentStatus: "PAID" }
D { createdAt: { $gte: start } }
E D again, key hashed customerId
```

YOUR TASKS

- 1 Classify queries A and B
- 2 Classify queries C and D
- 3 Classify E under the hashed key
- 4 Fix a slow monthly revenue report

SOLUTION (AFTER DEBRIEF)

- ✓ A, B targeted: key prefix
- ✓ C scatter-gather
- ✓ D many shards or all
- ✓ E scatter-gather
- ✓ Pre-aggregate, don't re-key

KEY TAKEAWAY The leading shard-key field decides routing — reports without it need a different design, not a new key.

LAB 6 — REPLICATION AND SHARDING

Day 3 hands-on: choose the mechanism, reason about shard keys, check a replica set.

OFFICIAL LAB

40 MIN

labs/day-03/lab6/LAB-6-GUIDE.md

SCENARIO

Steps 1–3 work on paper. Steps 4–7 need the instructor's replica-set or Atlas URI; a standalone mongod is not a replica set.

```
use training_store
rs.status()
rs.conf()
db.hello()
rs.printSecondaryReplicationInfo()
```

YOUR TASKS

- 1 Mechanism for six requirements
- 2–3 Shard keys and query routing
- 3 4 Confirm a replica set
- 4 5–6 Record members, votes and lag
- 5 7 Write a short health report

EXPECTED OUTCOME

- ✓ Last week's delete: a backup
- ✓ fulfillmentStatus rejected
- ✓ Status-only query scatters
- ✓ One PRIMARY, health: 1
- ✓ Lag measured, often 0–2 s

KEY TAKEAWAY Name the mechanism, defend the shard key, and report health as measured — not assumed.

MODULE 7 SUMMARY AND WHAT'S NEXT

From one replica set to a sharded cluster.

YOU CAN NOW

- ✓ Tell availability from scalability
- ✓ Explain the primary, secondaries, oplog and elections
- ✓ Choose read preference, read and write concern
- ✓ Explain replication lag and rollback
- ✓ Evaluate shard keys and query routing
- ✓ Decide when to replicate, shard, or both

NEXT: MODULE 8 — BEST PRACTICES

- 1 What will we actually run in production?
- 2 How do we secure, back up and restore it?
- 3 How do we monitor and troubleshoot it?

BRING ALONG Your Lab 6 health report and answers, and your shard-key scores from Exercise 7.3.

KEY TAKEAWAY Module 8 turns replication, sharding and backups into a production checklist.